Domain-grounded LLM agents for reliable orchestration of physics-constrained scientific workflows

Rongbo Shao[1], Guangzhi liao[1], Xingcai Zhang[2], Jun Zhou[3],

Juan Zhang[3], Xianzhi Song[1], Lizhi Xiao[1*]

[1]State Key Laboratory of Petroleum Resources and Engineering, China University of Petroleum, Beijing, China

[2]Department of Chemical and Nano Engineering, University of California, San Diego, La Jolla, CA, USA

[3]China National Logging Corporation, Shaanxi, China

[*]Corresponding author. Email: xiaolizhi@cup.edu.cn

Scientific workflows in data-intensive domains require more than accurate prediction models: they require sequential decision-making, heterogeneous tool coordination, physical-constraint checking, exception handling and reproducible reporting. Existing machine-learning systems usually automate isolated subtasks, leaving the orchestration of complete scientific workflows to human experts. Here we introduce Auto-Logging, a domain-grounded large-language-model agent framework for autonomous orchestration of physics-constrained scientific workflows, demonstrated in subsurface well-log interpretation. Auto-Logging separates workflow-level reasoning from numerical computation through a dual-agent architecture. A Planner agent interprets user intent, retrieves domain procedures and physical rules from a structured petrophysical knowledge base, and constructs task plans with explicit dependencies. An Executor agent grounds these plans by invoking specialized numerical models and preprocessing tools through stan-

dardized interfaces, returning structured feedback that enables dynamic workflow revision, exception handling and traceable report generation. We evaluate Auto-Logging on forward-simulated data and real well-log data from the SPWLA dataset, covering data quality control, missing-curve imputation, lithology classification, reservoir-parameter prediction, fluid identification and report generation. Compared with expert-assisted conventional interpretation, Auto-Logging reduces end-to-end interpretation time from hours to minutes, substantially decreases manual intervention, and improves run-to-run consistency while maintaining competitive or superior prediction accuracy for key reservoir parameters. Compared with strong task-specific AI baselines, the framework uniquely supports full end-to-end automation and dynamic multi-task coordination. Ablation experiments show that retrieval-augmented domain grounding, LLM-based scheduling and structured feedback loops each contribute to workflow reliability. These results establish a practical architecture for building scientific AI systems in which language models coordinate specialized computational components under explicit domain constraints. More broadly, Auto-Logging suggests a route toward reproducible, adaptive and human-auditable scientific workflow automation in engineering and data-intensive science.

1 Introduction

Complex scientific workflows increasingly combine high-dimensional data, specialized computational models, domain rules, physical constraints and iterative human decisions. Deep learning and foundation models have transformed single-task prediction across science and engineering, including protein structure prediction, materials discovery, large-scale drug discovery, medical-image segmentation, rare-disease model development, antimicrobial-peptide discovery and subcellular phenotype analysis (*1, 2, 3, 4, 5, 6, 7, 8*). At the same time, AI-accelerated discovery and machine-learning scientific systems are beginning to reshape climate prediction, engineering design and computational science more broadly (*9, 10*). However, the organization of complete scientific workflows remains heavily manual. Experts must decide which data-quality checks to run, which

models to invoke, how to respond to abnormal outputs, when to apply physical constraints and how to assemble results into reproducible reports. This gap between model-level automation and workflow-level autonomy limits scalability, consistency and rapid response in data-intensive scientific practice.

Recent progress also shows that AI and machine learning are no longer limited to conventional data classification; they are increasingly used to connect sensing, materials, biological and biomedical systems into data-driven discovery pipelines. Examples include learning-guided bioresource upgrading for sustainable energy, environment and biomedicine, AI-powered materials science, AI-enabled nanomedicine, and machine-learning microfluidic monitoring of microorganisms at minute-scale resolution (*11, 12, 13, 14*). These advances highlight both the breadth of AI-enabled scientific automation and the remaining need for reliable workflow-level systems that can coordinate data acquisition, model selection, physical or biological validation, exception handling and expert review.

Subsurface well-log interpretation provides a rigorous testbed for this broader problem. A complete interpretation workflow integrates in situ logging curves, geological context, petrophysical theory and engineering judgement to infer lithology, porosity, water saturation, shale volume and fluid type (*15, 16, 17, 18*). The workflow is sequential and strongly constrained: depth alignment and curve-quality verification must precede modeling; lithology constrains reservoir-parameter estimation; and reservoir parameters and resistivity jointly inform fluid interpretation. Errors in early stages propagate to downstream decisions, while anomalies such as missing curves, outliers, tool failures and physically implausible predictions require adaptive intervention. These characteristics make well-log interpretation an ideal domain for testing whether language-model agents can coordinate scientific workflows under explicit physical and procedural constraints.

Existing machine-learning approaches for well-log interpretation usually target isolated subtasks, such as lithology classification, missing-curve reconstruction or reservoir-parameter prediction (*17,18,19,20*). Similar model-centric automation has expanded rapidly in microfluidic sensing, virus detection, terahertz single-cell analysis, wearable monitoring and optical neural-computing platforms, where AI improves individual measurement or prediction stages but often still requires external workflow coordination (*21,22,23,24,25,26,27*). These models can be accurate within their task boundary, but they do not determine the correct sequence of operations, validate intermediate

outputs against physical rules, recover from tool failures or generate auditable end-to-end interpretation reports. General-purpose agent frameworks can call tools and decompose tasks, but they are not designed around petrophysical dependencies, mandatory data checks, physical plausibility constraints or field-deployment reliability (*28, 29, 30, 31, 32, 33, 34*). A scientific workflow agent must therefore combine language-based planning with domain grounding, structured execution and numerical models whose outputs remain traceable and verifiable.

Here we introduce Auto-Logging, a domain-grounded LLM-agent framework for reliable orchestration of physics-constrained scientific workflows. Auto-Logging uses a dual-agent architecture. A Planner agent performs intent understanding, retrieves petrophysical procedures and physical constraints through retrieval-augmented generation, and builds dependency-aware workflows (*35,36*). An Executor agent converts plans into structured tool and model invocations, manages data exchange through standardized interfaces, and returns execution summaries that enable adaptive revision (*28, 29*). The language model is not used as an unconstrained predictor; instead, it functions as an auditable workflow-level controller that coordinates specialized numerical components under domain rules.

This work makes four contributions. First, it introduces a Planner-Executor architecture that separates high-level scientific workflow planning from low-level numerical execution. Second, it shows how retrieval-augmented domain knowledge and physical constraints can make LLM planning more reliable in a specialized scientific setting. Third, it implements a closed-loop planning-execution-feedback mechanism that adapts workflows when data quality, model accuracy or tool execution fails. Fourth, it validates the framework on both forward-simulated and real well-log datasets, comparing it with expert-assisted interpretation, task-specific AI baselines and ablated system variants. Together, these results demonstrate a practical pathway for building scientific AI systems that are adaptive, reproducible and human-auditable (*37, 38*).

2 Results

2.1 A dual-agent architecture for domain-grounded workflow control

Auto-Logging transforms well-log interpretation from a fixed expert-operated pipeline into a knowledge-grounded agentic workflow, as illustrated in Fig 1. The system has two interacting agents. The Planner receives natural-language interpretation requests, retrieves relevant petrophysical rules and constructs a structured execution plan. The Executor converts this plan into model and tool invocations and returns structured results, quality indicators and exception signals. The separation of planning and execution makes each step inspectable: the reasoning-level decision, the numerical model invoked, the input-output schema and the validation result can all be logged independently (*28, 29, 34, 39*).

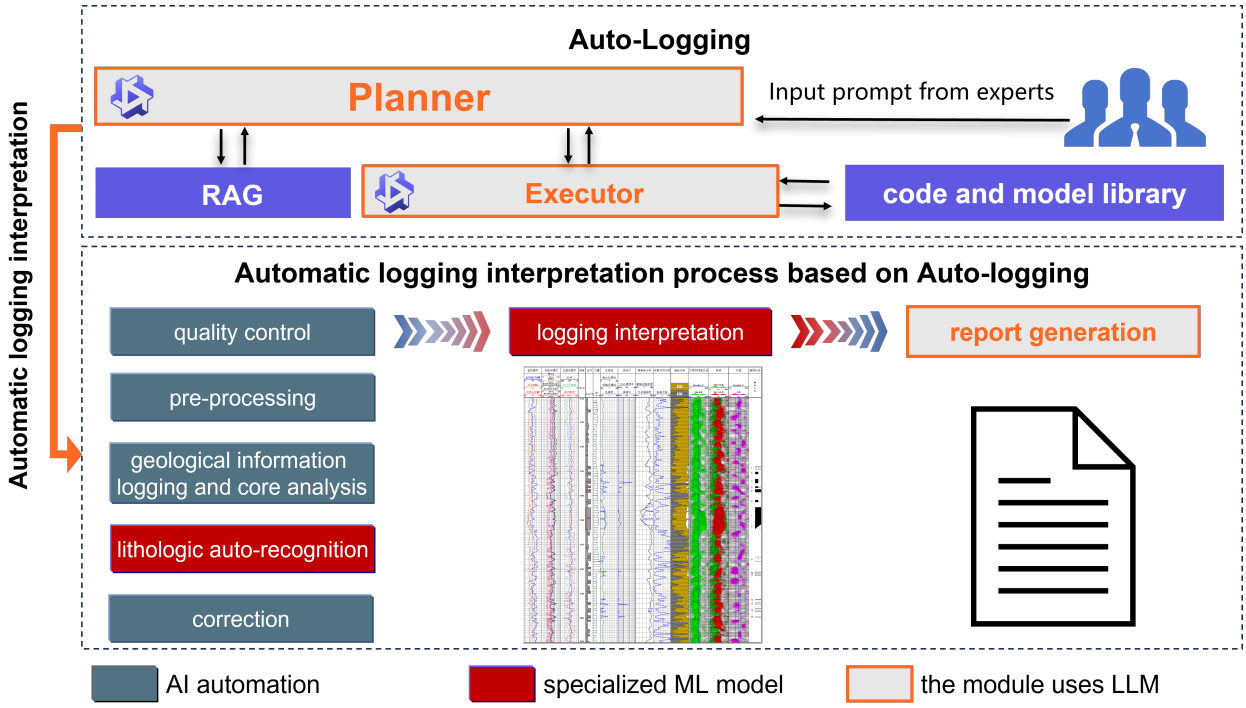


Figure 1: Overall architecture of Auto-Logging. The Planner uses retrieval-augmented domain knowledge to produce dependency-aware interpretation workflows. The Executor grounds these workflows by invoking preprocessing tools and specialized numerical models. Results are returned to the Planner for physical validation, adaptive revision and report generation.

The architecture differs from monolithic prediction systems in two ways. First, no single neural

network is expected to solve the entire interpretation task. Instead, task-specific models are encapsulated as independently callable components, including missing-curve imputation, lithology classification, reservoir-parameter prediction and fluid identification. Second, the LLM agent does not directly infer petrophysical quantities. It coordinates the correct tools, checks whether their outputs satisfy domain constraints and triggers recovery actions when outputs are incomplete, inconsistent or physically implausible.

2.2 Retrieval-augmented planning under physical and procedural constraints

Reliable scientific workflow orchestration requires that the agent know not only what tools exist, but also when they are scientifically appropriate. Auto-Logging therefore grounds the Planner through a structured petrophysical knowledge base containing workflow rules, physical thresholds, curve-response relationships, model requirements and report-generation constraints. During planning, retrieved knowledge is used to enforce the order of operations, select models compatible with the available curves, and define validation criteria for intermediate results (*16, 17, 35*).

For example, if a request asks for reservoir-parameter estimation from raw curves, the Planner first inserts data-quality control and preprocessing steps before any prediction model is called. If acoustic, neutron or resistivity curves have extended missing intervals, the workflow branches to missing-curve imputation before lithology or parameter inference. If porosity, shale volume or water saturation values exceed physically acceptable ranges, the Planner does not simply pass the result downstream; it requests reprocessing, switches to a backup model or asks for expert verification depending on the severity of the anomaly. This design shifts the role of the LLM from open-ended text generation to constrained scientific control.

2.3 Closed-loop execution and adaptive exception handling

Auto-Logging advances through a closed planning-execution-feedback loop (Fig 2). The Planner issues a structured task; the Executor converts it into a standardized command; the target tool or model performs the computation; the result is stored and summarized; and the Executor returns status, metrics and abnormal-condition flags to the Planner. The Planner then decides whether to continue, revise the workflow or trigger expert intervention. This loop provides the main mechanism

Table 1: Conceptual comparison between Auto-Logging and common AI workflow paradigms.

Dimension	Generic tool agents	Task-specific AI models	Auto-Logging
Primary role	General tool invocation and task decomposition	Accurate prediction for an isolated task	Domain-grounded orchestration of complete workflows
Domain constraints	Usually optional or prompt-level only	Embedded implicitly in training data	Explicitly retrieved, checked and logged
Workflow dependencies	Weakly enforced	Outside model scope	Encoded as mandatory step dependencies
Exception handling	Retry or stop	Usually unavailable	Tiered response: retry, workflow revision, backup model or expert review
Numerical prediction	May rely on the LLM or external tools	Performed by one model	Performed by specialized numerical models; LLM only coordinates
Traceability	Variable	Limited to model output	Plan, command, model result, validation and report are separately traceable

for adaptive control in the presence of incomplete data, model uncertainty, interface timeouts or physical inconsistency (*38*).

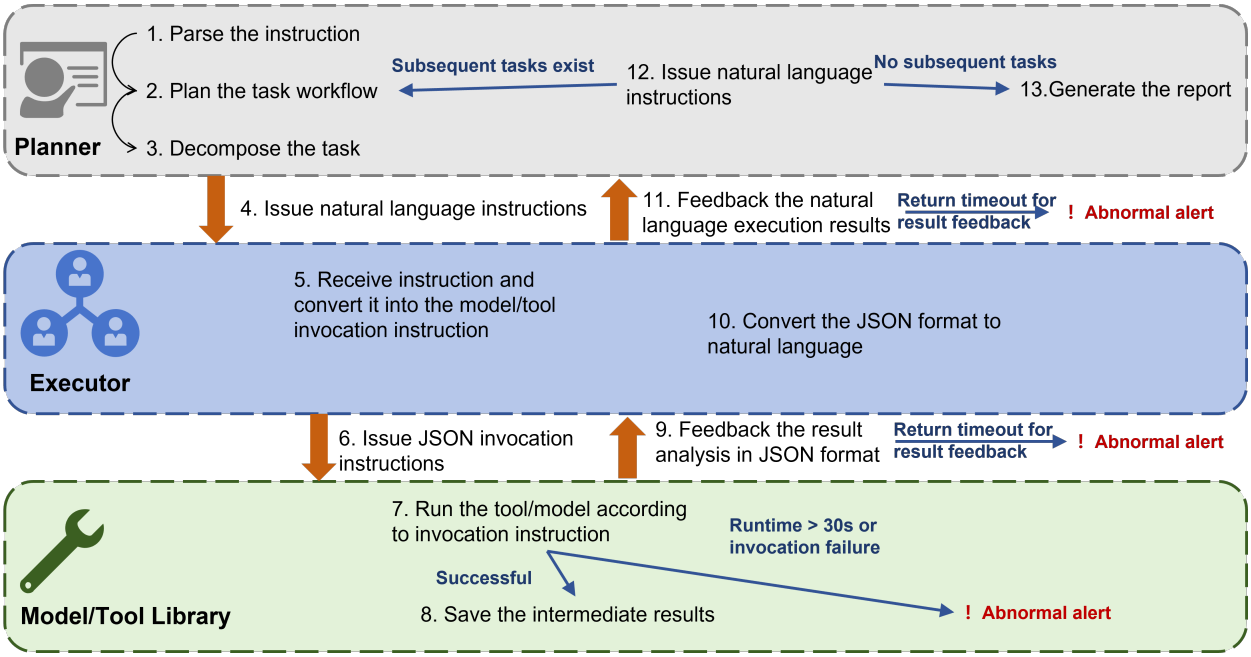


Figure 2: Dynamic adjustment and abnormal-alarm mechanism. The system iterates across Planner, Executor and model/tool library. Structured result feedback allows the Planner to continue the workflow after successful execution, revise the plan when moderate anomalies are detected or trigger expert review when critical failures occur.

Exceptions are grouped into four categories: data-related anomalies, model- or tool-related anomalies, communication anomalies and process-dependency conflicts. Minor failures, such as a transient timeout, trigger automatic retry. Moderate failures, such as prediction accuracy below a preset threshold or a violated physical bound, trigger workflow revision or backup-model selection. Critical failures, such as unavailability of a core component or repeated physical inconsistency, suspend autonomous execution and request human review. This tiered design is central to making the system suitable for scientific and engineering use, where silent failure is less acceptable than controlled escalation.

2.4 End-to-end interpretation workflow

The implemented workflow covers the main stages of well-log interpretation: data loading, quality control, preprocessing, missing-curve imputation, lithology classification, reservoir-parameter prediction, fluid identification, potential evaluation and report generation. Raw data are first checked against physical validity ranges and curve-continuity requirements. Sporadic outliers are corrected or clipped to valid bounds, while extended abnormal segments are marked for imputation. Bounded curves are normalized to a fixed interval, and resistivity curves are log-transformed to reduce the effect of multi-order magnitude variation (*17, 18*). The same principle of coupling AI with physically grounded measurement, transport or theranostic platforms has become increasingly important across nanomedicine, immunotherapy, nature-inspired functional materials and wireless therapeutic systems (*40, 41, 42, 43*).

Specialized numerical models then perform the core quantitative tasks. An LSTM-based model reconstructs missing AC, CNL and RT curves using complete auxiliary curves (*19*). A 2D-CNN identifies lithology from local depth windows and multiple logging responses (*1*). A multi-task Bi-LSTM predicts shale volume, porosity and water saturation (*15, 19, 44*). Fluid type is inferred from the combination of reservoir parameters and resistivity-related features. Finally, the Planner integrates results into a standardized report with explicit data provenance and interval-level interpretation (Fig. 3).

2.5 Datasets and evaluation design

We evaluated Auto-Logging using two datasets with complementary purposes (Table 2). The forward-simulated dataset was generated from petrophysical models to provide complete ground truth for full-process validation, including lithology, fluid type and multiple log curves under controlled noise. The real-world SPWLA competition dataset was used to test core task performance under realistic logging conditions. The combination allows evaluation of both workflow completeness and practical robustness.

Performance was assessed at three levels: workflow coordination, model prediction and system reliability. Workflow metrics included task-decomposition accuracy, invocation success, processing time, manual-intervention time and task completion rate. Prediction metrics included accuracy,

Table 2: Datasets used for evaluation.

Dataset	Samples / wells	Input curves	Targets	Primary purpose
Forward-simulated dataset	41,740 samples	AC, CAL, CNL, DEN, GR, RT, RXO, SP	Lithology, fluid type, Vsh, porosity, water saturation and related petrophysical quantities	End-to-end functional validation, ablation and controlled error analysis
SPWLA competition dataset	41,651 samples from 10 wells	CAL, DEN, GR, NEU, RMED, RDEP	PHIF, SW and VSH	Real-world validation of preprocessing and reservoir-parameter prediction

precision, recall, F1 score, mean absolute error, mean squared error and coefficient of determination. Reliability analyses included ablations, manual-expert comparison, state-of-the-art model comparison and qualitative error analysis.

2.6 System-level automation and efficiency

On the forward-simulated dataset, the Planner correctly decomposed representative user requests into valid workflows and integrated approximately 1,500 interval-level data points into report-ready outputs. The Executor achieved high model-invocation success and completed a single-well interpretation in approximately 25 minutes. On the SPWLA dataset, the Planner generated a complete workflow for a roughly 2,000-sample well in seconds, and the full interpretation finished in approximately 10 minutes without manual intervention.

Deployment tests on a cloud environment showed stable operation under repeated execution(Table 3). Task planning required approximately 12-18 s, single-well interpretation required approximately 15-30 min depending on data volume and available curves, and batch processing

of 10 wells scaled approximately linearly. A 48-hour continuous operation test achieved a task-completion rate of 99.7%, and common input formats such as LAS, CSV and TXT were supported with high loading success. These results indicate that the main advantage of Auto-Logging is not only predictive accuracy but the ability to coordinate a complete, auditable workflow.

Table 3: System-level automation and efficiency metrics.

Metric	Auto-Logging result	Interpretation
Planner workflow generation	15.4 ± 1.2 s on a representative SPWLA well	Rapid construction of dependency-aware workflows
Full SPWLA well interpretation	10.2 ± 1.3 min	End-to-end execution without manual intervention in the tested case
Forward-simulated single-well interpretation	25.3 ± 2.1 min	Full-process workflow including reporting
Continuous operation task completion	99.7% over 48 h	Stable cloud-based execution
Manual intervention	$\tilde{1}$ min in benchmark comparison	Substantially reduced repetitive expert operation

2.7 Prediction performance on simulated and real data

On the forward-simulated dataset, missing-curve imputation reconstructed AC, CNL and RT curves with strong agreement to the held-out ground truth. The best imputation performance was obtained for CNL, while RT remained more difficult because of its high dynamic range and sensitivity to fluid and borehole conditions. Lithology classification achieved high overall performance across four lithology types, with particularly strong results for shale and fine sandstone. Multi-task reservoir-parameter prediction produced high R^2 values for shale volume, porosity and water saturation, and fluid identification reached high accuracy across five fluid classes (Table 4).

On the SPWLA real-world dataset, preprocessing reduced the outlier ratio from 3.2% to 0.8% and achieved a 99.1% curve-continuity compliance rate. Reservoir-parameter prediction applied

without task-specific retuning achieved R^2 values above practical thresholds for VSH, PHIF and SW. Rule-based fluid judgement using the predicted parameters achieved 91.3% accuracy in distinguishing oil layers, water layers and non-reservoir intervals, while reservoir-potential evaluation reached 85.6% consistency with expert annotations.

Table 4: Summary of predictive performance.

Task	Dataset	Main result	Comment
Missing-curve imputation	Forward simulation	CNL R^2 = 0.9896 ± 0.003; AC R^2 = 0.8992 ± 0.012; RT R^2 = 0.7906 ± 0.018	RT is more difficult because resistivity varies over multiple orders of magnitude
Lithology identification	Forward simulation	Overall accuracy = 91.5%; shale F1 = 0.96 ± 0.01; fine sandstone F1 = 0.92 ± 0.02	Strong performance on dominant lithology classes
Reservoir-parameter prediction	Forward simulation	Vsh R^2 = 0.9959 ± 0.0012; POR R^2 = 0.9392 ± 0.0087; SW R^2 = 0.9477 ± 0.0073	High accuracy across all three core reservoir parameters
Reservoir-parameter prediction	SPWLA	VSH R^2 = 0.9287 ± 0.012; PHIF R^2 = 0.8863 ± 0.015; SW R^2 = 0.8945 ± 0.013	Meets the practical threshold of $R^2 \geq$ 0.85
Fluid judgment	SPWLA	Accuracy = 91.3%; reservoir-potential consistency = 85.6%	Uses predicted parameters and domain rules

2.8 Comparison with expert-assisted interpretation and AI baselines

We compared Auto-Logging with an expert-assisted conventional workflow in which three senior logging interpreters used industry-standard commercial software. Auto-Logging reduced total interpretation time from approximately 14 h to 25.3 min, increased consistency across repeated interpretations, and greatly reduced manual intervention. The gain should not be interpreted as replacing geological experts; rather, it indicates that repetitive workflow execution can be automated while experts retain responsibility for verification, high-stakes decisions and edge-case review.

We also compared Auto-Logging with task-specific AI baselines retrained under the same hardware conditions, including Transformer-based, graph-based, CNN-LSTM and MLP-ensemble models for logging interpretation (*1, 18, 19, 20*).Some single-task baselines achieved slightly better performance for an individual parameter, but they did not provide complete workflow coordination, adaptive exception handling or standardized report generation. Auto-Logging therefore offers a different advantage: balanced multi-task performance combined with end-to-end automation and traceable workflow control (Table 5).

2.9 Ablation analysis identifies the contribution of domain grounding and adaptive control

Ablation experiments confirmed that Auto-Logging's performance depends on the interaction of domain grounding, LLM scheduling and adaptive feedback (Table 6. Removing retrieval-augmented generation reduced task compliance and lowered average parameter-prediction reliability, showing that workflow planning benefits from explicit petrophysical knowledge rather than generic reasoning alone. Replacing adaptive planning with a fixed pipeline sharply reduced task-decomposition accuracy and approximately doubled processing time, demonstrating that workflow adaptation is necessary when data conditions vary. Replacing the larger Planner with a smaller model preserved basic execution ability but reduced performance on complex reasoning and fine-grained workflow revision (*36, 38, 45*).

Table 5: Practical comparison between Auto-Logging and alternative interpretation approaches.

Comparator	Strength	Limitation relative to Auto-Logging
Expert-assisted conventional workflow	High domain expertise and flexible judgement	Slow, labor-intensive and less reproducible across interpreters
Transformer-Log	Strong single-task PHIF prediction	No complete workflow control or dynamic exception handling
GNN-Petrophysics	Uses relationships among curves	Requires external orchestration and reporting
CNN-LSTM Hybrid	Captures spatial and sequential features	Limited end-to-end automation
MLP-Ensemble	Simple and stable baseline	Weak workflow adaptivity and limited physical validation
Auto-Logging	End-to-end orchestration, physical validation, adaptive control and report generation	Requires curated domain knowledge base and reliable component registry

2.10 Error analysis

Two important discrepancy patterns were observed in the SPWLA experiments. First, some intervals showed disagreement between predicted porosity and reference labels when the neutron porosity curve was high and the density curve was low. This combination is physically consistent with high porosity, whereas the reference labels indicated low porosity. The discrepancy may therefore reflect label limitations rather than model failure. This illustrates a useful property of physics-constrained agents: they can expose potentially inconsistent labels when model outputs better align with physical curve relationships.

Second, VSH predictions were accurate in shallow intervals but underestimated in some deeper

Table 6: Ablation summary.

Variant	Observed effect	Interpretation
No RAG	Task-decomposition accuracy decreased and average parameter R2 declined	Knowledge grounding improves procedural correctness and physical compliance
Fixed pipeline / no adaptive planning	Task-decomposition accuracy dropped markedly; processing time increased to 45.2 min	Dynamic workflow revision is central to reliability and efficiency
Smaller Planner model	Acceptable average R2 but weaker complex reasoning	Large-model capacity is most valuable in edge cases and multi-step revisions
Full Auto-Logging	Best balance of automation, accuracy and reliability	Benefits come from system-level coordination, not a single model alone

intervals. Possible causes include changes in shale mineral composition that weaken gamma-ray response and insufficient representation of deep-interval samples in the training set. These cases show that Auto-Logging should be deployed as a human-auditable decision-support system rather than as an unchecked autonomous authority. When geological settings fall outside the coverage of the knowledge base or training data, the system should flag uncertainty and escalate to experts (Fig. 4).

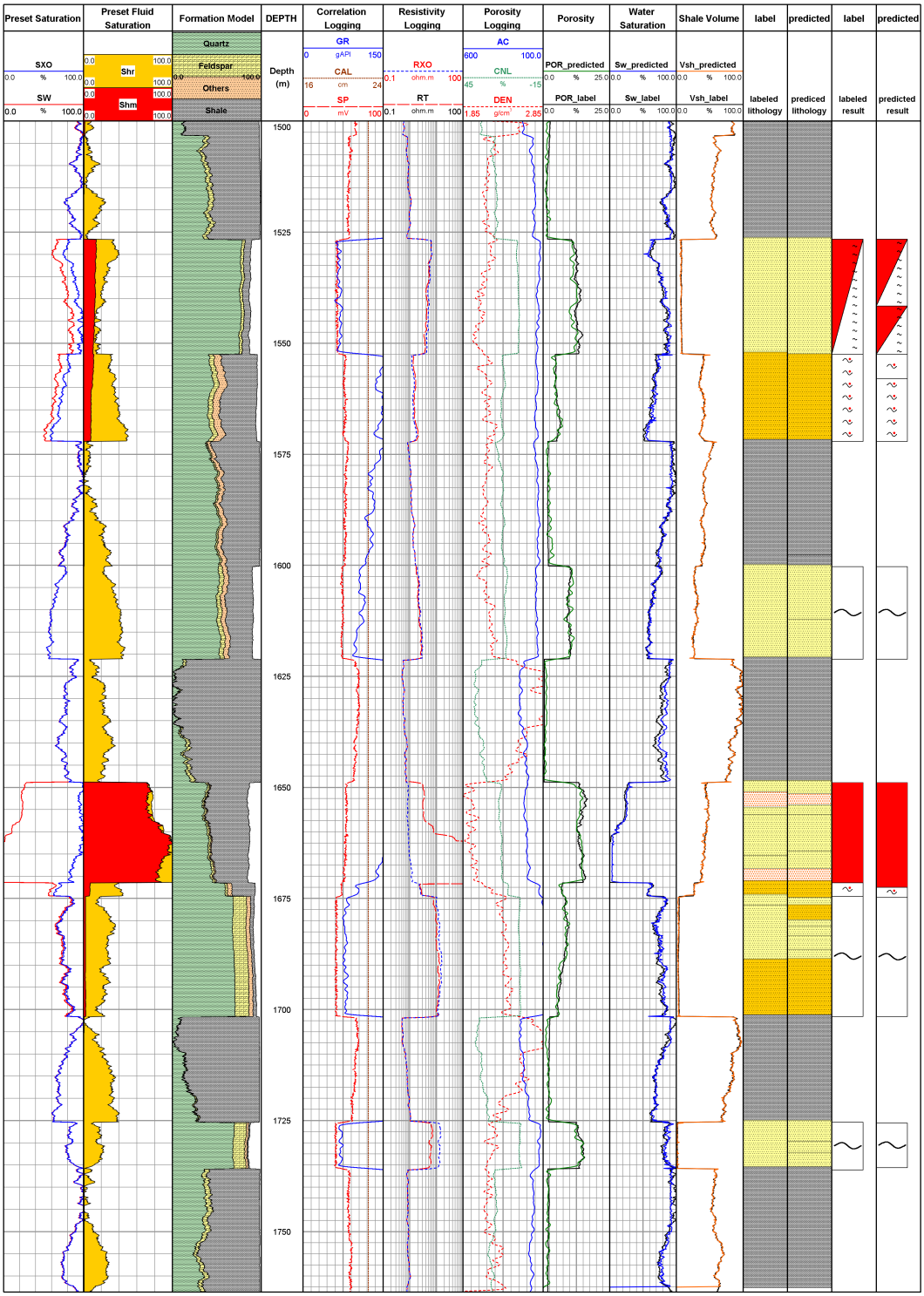


Figure 3: Example Auto-Logging interpretation result on forward-simulated data. The output integrates curve tracks, lithology, reservoir-parameter predictions and interval-level interpretation results into a report-ready well-log display.

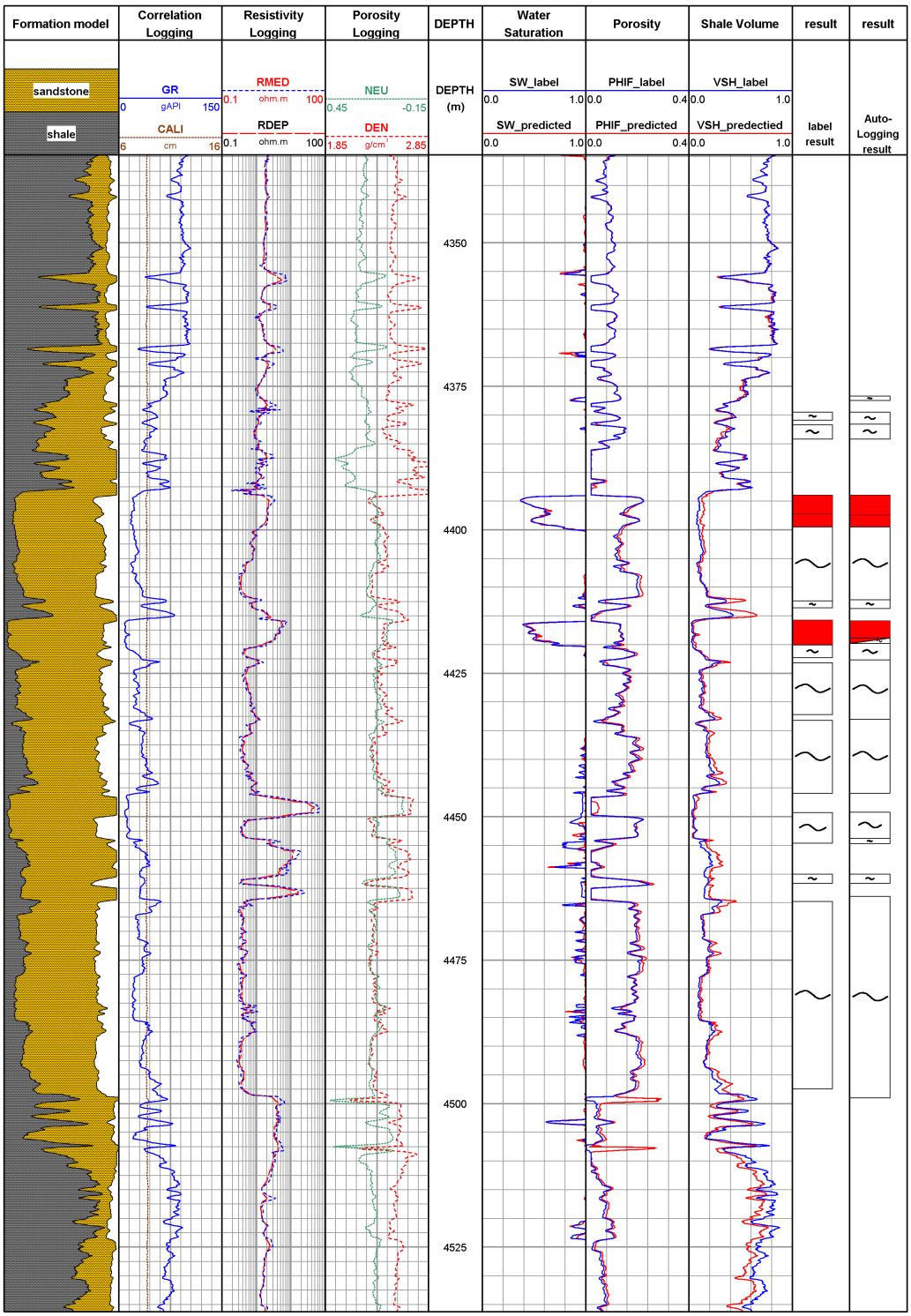


Figure 4: Example Auto-Logging interpretation result on the SPWLA real-world dataset. The figure illustrates integrated curve tracks, predicted reservoir parameters and interval-level interpretation outputs under realistic logging conditions.

3 Discussion

Auto-Logging demonstrates that language-model agents can contribute to scientific automation when their role is carefully constrained (*34, 38*). The LLM agents do not replace numerical models and do not directly produce petrophysical predictions. Instead, they coordinate specialized components through explicit task plans, structured interfaces and physical validation rules. This design addresses a central challenge in scientific AI: moving from isolated prediction models to complete workflows that are reproducible, adaptive and auditable.

The most important result is therefore architectural rather than only numerical. Auto-Logging shows how domain knowledge, tool orchestration, exception handling and reporting can be integrated into a single closed-loop system. In this framework, expert knowledge is not discarded; it is formalized as retrievable procedures, physical thresholds, dependency rules and validation criteria (*16, 37*). Experts remain essential for defining the knowledge base, reviewing abnormal cases, correcting new geological scenarios and making final operational decisions.

The study also clarifies the boundary between general-purpose AI agents and domain-grounded scientific agents. A general agent can decompose instructions and call tools, but scientific workflow execution requires a stronger contract: task dependencies must be explicit, inputs and outputs must follow validated schemas, numerical results must satisfy physical constraints, and failures must be escalated rather than hidden (*30, 31, 34*). Auto-Logging implements this contract in well-log interpretation and thereby provides a reusable design pattern for other data-intensive domains, including geophysical inversion, environmental monitoring, materials characterization, biomedical image analysis, bioresource upgrading, AI-powered materials science, nanomedicine and machine-learning microfluidic monitoring (*3, 10, 11, 12, 13, 14, 44*).

Several limitations remain. First, generalization depends on the coverage of the knowledge base and the model library. Extending Auto-Logging to carbonate reservoirs, shale gas reservoirs or other geological settings will require additional domain rules, calibration data and validation cases. Second, LLMs can still make reasoning errors in rare or ambiguous conditions, so audit logs and expert-review triggers remain essential. Third, cloud-based deployment provides scalability but introduces latency, cost and data-governance considerations. Fourth, the present validation focuses on well-log interpretation; broader claims about scientific workflow automation should be

supported by additional cross-domain demonstrations.

Future work should therefore expand the knowledge base, add uncertainty quantification to every tool invocation, test well-level transfer across geological settings, implement stronger provenance tracking, and benchmark against additional agentic baselines. A particularly important direction is continual learning under expert supervision, where corrected edge cases are used to update the knowledge base and improve future workflow decisions. With these extensions, domain-grounded LLM agents could become a practical foundation for reliable and human-auditable scientific workflow automation.

4 Methods

4.1 System overview

Auto-Logging contains four layers: an LLM-based Planner, an LLM-based Executor, a model/tool library and a cloud service layer. The Planner performs intent parsing, knowledge retrieval, workflow construction and adaptive revision. The Executor converts tasks into structured commands, invokes models or tools, verifies returned results and reports execution status. The model/tool library contains preprocessing utilities and specialized numerical models. The cloud service layer provides standardized APIs, storage, authentication and deployment management.

4.2 Prompt and retrieval design

Planner and Executor prompts use a modular structure containing role definition, task specification, domain constraints, output schemas and examples. The Planner prompt emphasizes petrophysical workflow logic, dependency ordering and physical validation. The Executor prompt emphasizes parameter formatting, model invocation, result checking and exception feedback. Retrieval-augmented generation uses a structured knowledge base compiled from petrophysical procedures, standard log-interpretation workflows, curve-response relationships and parameter validity rules (*16, 35*). The evaluated implementation used a versioned instruction-tuned LLM backbone, with decoding settings, prompt version and model identity recorded for each run (*36, 38, 45*).

4.3 Preprocessing

Preprocessing includes data loading, curve validation, outlier detection, missing-segment localization, depth-consistency checking, curve normalization and transformation of high-dynamic-range resistivity curves. Sporadic outliers are corrected while extended missing or abnormal segments are passed to the imputation model. All preprocessing outputs are stored with metadata that link each corrected interval to the operation that produced it.

4.4 Numerical models

Missing-curve imputation is performed by an LSTM-based sequence model using complete auxiliary curves as input. Lithology identification uses a 2D convolutional neural network over local depth windows and multiple curve features. Reservoir-parameter prediction uses a multi-task recurrent network with shared sequential representation and task-specific heads for shale volume, porosity and water saturation. Fluid identification combines predicted parameters, resistivity features and domain rules or classification models depending on the available labels.

4.5 Workflow execution

For each user request, the Planner generates a structured task plan with dependencies, required inputs, target tools or models and expected validation criteria. The Executor checks whether required inputs are available, formats requests according to the component registry, invokes the corresponding service and returns structured execution summaries. The Planner decides whether the workflow can continue, should be revised or requires expert intervention.

4.6 Evaluation

Evaluation used forward-simulated and SPWLA real-world well-log data. The forward-simulated dataset supported full-process validation because all target quantities were known. The SPWLA dataset tested performance on realistic curves and interpretation labels. We evaluated predictive performance using accuracy, precision, recall, F1 score, MAE, MSE and R2, and evaluated workflow

performance using processing time, task-decomposition accuracy, invocation success, manual-intervention time and completion rate.

4.7 Expert and baseline comparison

For the expert-assisted baseline, senior interpreters completed the same interpretation tasks using commercial software. For AI baselines, representative task-specific models were trained or retrained under comparable hardware conditions. Comparisons emphasized not only single-task accuracy but also workflow completeness, dynamic coordination capability, automation level and report-generation ability.

4.8 Ablation experiments

Ablations removed retrieval-augmented generation, replaced adaptive planning with a fixed pipeline, or replaced the larger Planner model with a smaller model. Each variant was evaluated on task-decomposition accuracy, model-scheduling success, processing time, parameter-prediction performance and manual-intervention frequency.

4.9 Statistical analysis

Reported values are presented as mean ± standard deviation where repeated tests were available. R2, MAE and MSE were computed on held-out test intervals or wells according to the evaluation setting. Classification metrics were computed per class and as overall task accuracy. Consistency between repeated interpretations was assessed using intraclass correlation when applicable.

References and Notes

1. Y. LeCun, Y. Bengio, G. Hinton, Deep learning. *Nature* **521** (7553), 436–444 (2015).

2. J. Jumper, *et al.*, Highly accurate protein structure prediction with AlphaFold. *Nature* **596** (7873), 583–589 (2021).

3. A. Merchant, *et al.*, Scaling deep learning for materials discovery. *Nature* **624** (7990), 80–85 (2023).

4. M. Yu, *et al.*, Deep learning large-scale drug discovery and repurposing. *Nature Computational Science* **4**, 600–614 (2024).

5. S. Zhang, *et al.*, A generalist foundation model and database for open-world medical image segmentation. *Nature Biomedical Engineering* pp. 1–16 (2025).

6. F. Dai, *et al.*, Improving AI models for rare thyroid cancer subtype by text guided diffusion models. *Nature Communications* **16** (1), 4449 (2025).

7. J. Wang, *et al.*, Discovery of antimicrobial peptides with notable antibacterial potency by an LLM-based foundation model. *Science advances* **11** (10), eads8932 (2025).

8. X. Zhang, Y. Wang, AI-recognized mitochondrial phenotype enables identification of drug targets (2024).

9. I. Price, *et al.*, Probabilistic weather forecasting with machine learning. *Nature* **637** (8044), 84–90 (2025).

10. R. Lam, *et al.*, Learning skillful medium-range global weather forecasting. *Science* **382** (6677), 1416–1421 (2023).

11. F. Li, *et al.*, Bioresource upgrade for sustainable energy, environment, and biomedicine. *Nano-Micro Letters* **15** (1), 35 (2023).

12. X. Bai, X. Zhang, Artificial intelligence-powered materials science. *Nano-micro letters* **17** (1), 135 (2025).

13. G. Luo, *et al.*, Artificial intelligence-powered nanomedicine. *Chemical Society Reviews* (2026).

14. N. Yang, *et al.*, Machine-Learning Microfluidic Minute-Scale Microorganism Metrics Monitoring (M6). *Advanced Science* p. e21106 (2026).

15. C. Shen, *et al.*, Differentiable modelling to unify machine learning and physical models for geosciences. *Nature Reviews Earth & Environment* **4** (8), 552–567 (2023).

16. D. V. Ellis, J. M. Singer, *et al.*, *Well logging for earth scientists*, vol. 692 (Springer) (2007).

17. S. Misra, H. Li, J. He, *Machine learning for subsurface characterization* (Gulf Professional Publishing) (2019).

18. X. Wu, *et al.*, Sensing prior constraints in deep neural networks for solving exploration geophysical problems. *Proceedings of the National Academy of Sciences* **120** (23), e2219573120 (2023).

19. S. Hochreiter, J. Schmidhuber, Long short-term memory. *Neural computation* **9** (8), 1735–1780 (1997).

20. A. Vaswani, *et al.*, Attention is all you need. *Advances in Neural Information Processing Systems* **30** (2017).

21. B. Wang, *et al.*, Smartphone-based platforms implementing microfluidic detection with image-based artificial intelligence. *Nature Communications* **14** (1), 1341 (2023).

22. T. Li, *et al.*, Virus detection light diffraction fingerprints for biological applications. *Science Advances* **10** (11), eadl3466 (2024).

23. N. Yang, *et al.*, Minimum minutes machine-learning microfluidic microbe monitoring method (M7). *ACS nano* **18** (6), 4862–4870 (2024).

24. N. Yang, *et al.*, Deep-learning terahertz single-cell metabolic viability study. *ACS nano* **17** (21), 21383–21393 (2023).

25. L. Liu, *et al.*, Artificial intelligence-powered microfluidics for nanomedicine and materials synthesis. *Nanoscale* **13** (46), 19352–19366 (2021).

26. H. Liu, *et al.*, Machine-learning mental-fatigue-measuring μm-thick elastic epidermal electronics (MMMEEE). *Nano Letters* **24** (51), 16221–16230 (2024).

27. X. Wang, P. Xie, B. Chen, X. Zhang, Chip-based high-dimensional optical neural network. *Nano-Micro Letters* **14** (1), 221 (2022).

28. D. A. Boiko, R. MacKnight, B. Kline, G. Gomes, Autonomous chemical research with large language models. *Nature* **624** (7992), 570–578 (2023).

29. A. M. Bran, *et al.*, Augmenting large language models with chemistry tools. *Nature Mchine Intelligence* **6** (5), 525–535 (2024).

30. V. Mavroudis, LangChain v0. 3. *GitHub repository* (2024).

31. G. Significant, AutoGPT. *GitHub repository* (2023).

32. K. Swanson, W. Wu, N. L. Bulaong, J. E. Pak, J. Zou, The Virtual Lab of AI agents designs new SARS-CoV-2 nanobodies. *Nature* **646** (8085), 716–723 (2025).

33. N. J. Szymanski, *et al.*, An autonomous laboratory for the accelerated synthesis of inorganic materials. *Nature* **624** (7990), 86 (2023).

34. H. Xin, J. R. Kitchin, H. J. Kulik, Towards agentic science for advancing scientific discovery. *Nature Machine Intelligence* **7** (9), 1373–1375 (2025).

35. J. Dagdelen, *et al.*, Structured information extraction from scientific text with large language models. *Nature communications* **15** (1), 1418 (2024).

36. J. Wei, *et al.*, Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* **35**, 24824–24837 (2022).

37. A. Clark, Extending minds with generative AI. *Nature Communications* **16** (1), 4627 (2025).

38. L. Zhou, *et al.*, Larger and more instructable language models become less reliable. *Nature* **634** (8032), 61–68 (2024).

39. B. Romera-Paredes, *et al.*, Mathematical discoveries from program search with large language models. *Nature* **625** (7995), 468–475 (2024).

40. X. Li, *et al.*, Transvascular transport of nanocarriers for tumor delivery. *Nature communications* **15** (1), 8172 (2024).

41. Z. Yang, *et al.*, Thermal immuno-nanomedicine in cancer. *Nature reviews Clinical oncology* **20** (2), 116–134 (2023).

42. Y. Wang, *et al.*, Nature-inspired micropatterns. *Nature Reviews Methods Primers* **3** (1), 68 (2023).

43. Y. Yang, *et al.*, A non-printed integrated-circuit textile for wireless theranostics. *Nature communications* **12** (1), 4876 (2021).

44. K. R. Lennon, G. H. McKinley, J. W. Swan, Scientific machine learning for modeling and simulating complex fluids. *Proceedings of the National Academy of Sciences* **120** (27), e2304669120 (2023).

45. . Qwen Team, *et al.*, Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

Acknowledgments

Funding: This work is supported by the National Science and Technology Major Project for New Oil and Gas Exploration and Development (Grant No. 2025ZD1401102).

Author contributions: The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

- Rongbo Shao: Conceptualization, methodology, software, validation, formal analysis, writing (original draft), visualization.
- Guangzhi Liao: Validation, formal analysis, writing (review & editing), supervision.
- Xingcai Zhang: Methodology guidance, Validation, formal analysis, writing (review & editing).
- Jun Zhou: Resources, data curation, validation, writing (review & editing), project administration.

- Juan Zhang: Resources, data curation, validation, writing (review & editing), funding acquisition.
- Xianzhi Song: Conceptualization, methodology guidance.
- Lizhi Xiao: Conceptualization, methodology guidance, supervision, writing (review & editing), funding acquisition.

Competing interests: The authors declare no competing interests.

Data, code and materials availability: The datasets and source code of this study are available at github

AI Declaration: In the development of this study, generative large language models (LLMs) were not used for core research components, including the programming and training of lightweight domain models, the design of the overall experimental workflow, or the formulation of prompt engineering strategies.

For the construction of the agent-based framework, two open-source LLMs (Qwen 7B and Qwen 32B) were employed solely as functional modules to support high-level workflow reasoning and planning, which forms an integral part of the proposed Auto-Logging system. Importantly, these LLMs were not involved in any numerical computations, such as reservoir parameter prediction, lithology classification, or fluid identification—all such professional numerical tasks were carried out by dedicated domain-specific models.

During manuscript preparation, the overall research ideas, article structure, experimental methodologies, and figure design were independently conceived and completed by the authors. Generative AI was used only for minor language polishing to enhance the clarity and fluency of academic expression, without altering the scientific content or conclusions. All references cited in the manuscript are verifiable and have been properly acknowledged in accordance with academic standards.

Supplementary materials

Materials and Methods

Supplementary Text

Figs. S1 to S9

Tables S1 to S9

Supplementary Materials for

Domain-grounded LLM agents for reliable orchestration of physics-constrained scientific workflows

Rongbo Shao, Guangzhi liao, Xingcai Zhang Jun Zhou, Juan Zhang, Xianzhi Song, Lizhi Xiao*

*Corresponding author. Email:xiaolizhi@cup.edu.cn

This PDF file includes:

Structured Prompt Frameworks

This section elaborates on the granular design of the prompt engineering framework underpinning the Planner and Executor agents, providing the technical specifications that enable reliable, domain-aligned LLM behavior.

Core Prompt Architecture

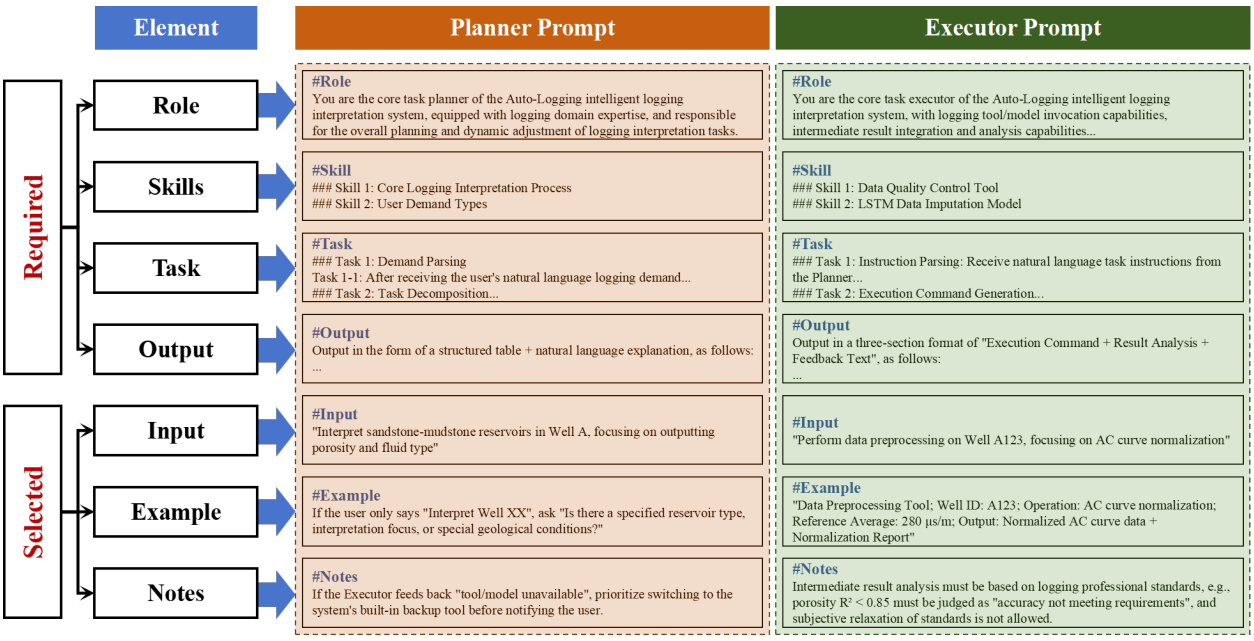


Figure S1: Prompt Engineering Elements and Examples of Prompt Writing for the Planner and Executor.

As illustrated in Fig. S1, each agent's prompt is constructed using a modular, two-tier structure that separates mandatory functional elements from optional performance-enhancing components, ensuring consistency while allowing adaptability to complex geoscientific tasks. **Mandatory Prompt Elements**

These foundational components establish the agent's functional identity and enforce task alignment:

1. Role Definition: Explicitly frames the agent's domain-specific persona to anchor LLM reasoning. The Planner is defined as a "senior logging interpretation workflow architect" with

expertise in petrophysical logic and dynamic workflow adjustment, while the Executor is designated a ”precision tool orchestrator” specializing in model invocation and result validation.

2. Skill Enumeration: Outlines technical competencies tailored to the agent's role. The Planner's skills include geological context integration, task dependency mapping, and adaptive workflow revision, whereas the Executor's skills focus on API orchestration, numerical result verification, and error diagnostics.

3. Task Specification: Articulates action-oriented objectives with clear input-output requirements. The Planner's task is to decompose user requests into physics-compliant, stepwise workflows, while the Executor's task is to translate these workflows into executable commands and validate outputs against petrophysical constraints.

4. Output Schema: Mandates adherence to a machine-parsable JSON format with strict field validation. For the Planner, outputs require structured task sequences with unique Instruction_ID, Task_Params, and Timestamp fields. The Executor's outputs include command logs, result metrics, and error codes aligned with the system's taxonomy (Init/Execute/Feedback-/Adjust/Alarm).

Optional Performance-Enhancing Components

These supplementary elements reduce ambiguity and improve robustness in high-complexity scenarios:

1. Contextual Input: Provides well-specific metadata (e.g., wellbore ID, log curve availability) to ground the agent's reasoning in real geological data.

2. Annotated Examples: Includes 2–3 few-shot demonstrations of valid task decomposition and command generation, reinforcing compliance with JSON schemas and domain constraints.

3. Domain Constraints: Embeds petrophysical validation rules (e.g., confidence_threshold between 0.5–0.9, curve_list as a standardized log curve abbreviation array) to ensure outputs align with industry standards.

Agent-Specific Prompt Optimization

Planner Prompts

Planner prompts prioritize cognitive coherence and scientific validity, with embedded mechanisms to mitigate LLM hallucinations:

1. Schema Enforcement: Prompts include full JSON schema definitions to enable self-validation of output structure, prohibiting unstructured text and ensuring compatibility with the Executor's command parser.

2. Dependency Awareness: Explicitly encodes petrophysical dependencies (e.g., lithology classification preceding porosity prediction) into prompt instructions, ensuring workflows adhere to geological reasoning.

Executor Prompts

Executor prompts focus on precision and reliability, with built-in error handling to maintain workflow continuity:

1. Error Diagnostics: Prompts include templates for structured error messages (e.g., "Missing Instruction_ID field") that trigger targeted Planner corrections at Level 1 of the fallback mechanism.

2. Dependency Awareness: Explicitly encodes petrophysical dependencies (e.g., lithology classification preceding porosity prediction) into prompt instructions, ensuring workflows adhere to geological reasoning.2.Result Validation: Mandates cross-checking of numerical outputs against physical constraints (e.g., porosity values between 0–1), with automated feedback to the Planner for parameter adjustments if thresholds are violated.

This modular framework ensures that prompts are both domain-informed and functionally robust, providing a replicable template for building AI agents in knowledge-intensive engineering applications. The explicit separation of mandatory and optional components balances standardization with flexibility, enabling the system to adapt to evolving geological scenarios while maintaining industrial-grade reliability.

Prompt for the Planner

This is the English translation of the original Chinese prompt for the Planner, preserving its domain constraints and structural design.

Role Definition

You are the core task planning component of the Auto-Logging intelligent logging interpretation system, equipped with logging domain expertise (covering reservoir interpretation logic for sandstone-mudstone and other lithologies, standard logging interpretation workflows), strong natural language understanding capabilities, chain-of-thought reasoning capabilities, and contextual dynamic adaptation capabilities. Guided by user needs and based on professional standards, you are responsible for the overall planning and dynamic adjustment of logging interpretation tasks.

Context and Skill Background

Skill 1: **Core Logging Interpretation Workflow**

You must proficiently master the full-link links of logging interpretation, including but not limited to:

Data quality control (outlier detection, curve consistency verification);

Data preprocessing (bad value removal, curve normalization, depth correction);

Automatic stratification (formation interface identification based on GR and AC curves);

Professional task execution (data imputation, lithology identification, reservoir parameter prediction, fluid identification);

Interpretation report generation (stratification results, parameter tables, conclusions and recommendations).

Skill 2: **User Demand Types**

You can handle simple demands (e.g., ”Complete full-process logging interpretation for Well XX”) and complex demands (e.g., ”For high-salinity sandstone-mudstone reservoirs in Block XX, focus on interpreting porosity and oil saturation, and ignore dry layer analysis”). You must accurately capture core information such as ”well ID, reservoir type, interpretation focus, and special constraints”.

Skill 3: **Feedback Adaptation Rules** You can receive intermediate results fed back by the Executor (e.g., ”3 outliers still exist in the AC curve after data preprocessing”, ”The accuracy

of the lithology identification model for siltstone is 82%") and adjust the plan based on logging professional logic (e.g., adding a "manual outlier review step", "supplementing siltstone logging response feature samples to the model").

Skill 4: **Interpretation Report Generation**

You can generate interpretation reports in accordance with the following formats and requirements:

- Paragraph 1: <Geological Background>

— Writing Requirements: Based on the <Oilfield Name> and <Oil-Gas Exploration Target Layer> in the input JSON format, use your existing knowledge to write a paragraph including the location of the oilfield, the lithological composition, sedimentary continuity, porosity, and permeability of the oil-gas exploration target layer.

- Paragraph 2: <Logging Data Processing>

— Writing Requirements: Based on the corresponding model names of <Porosity Model>, <Shale Content Model>, and <Water Saturation Model> in the input JSON format, output according to the following template:

—- Template:

Shale Content (VSH) Calculation: Shale content is mainly calculated using Gamma Ray (GR) and Photoelectric Factor (PEF), and corrected by Caliper (CAL). In this report, the Shale Content Model is adopted to predict shale content based on conventional logging curves.

Porosity (PHIF) Calculation: Porosity is mainly calculated using Density Logging (DEN) and Neutron Logging (NEU), and then corrected by shale content. In this report, the Porosity Model is adopted to predict porosity based on conventional logging curves.

Water Saturation (SW) Calculation: Water saturation is mainly calculated using Deep Resistivity (RDEP) and Medium Resistivity (RMED), and then corrected by porosity and shale content. In this report, the Water Saturation Model is adopted to predict water saturation based on conventional logging curves.

- Paragraph 3: <Oil-Gas-Water Evaluation of Logging Data>

— Writing Requirements: Based on the input and output parameter data of each stratified layer in the input JSON format, use Skill 1 to judge parameter levels and Skill 2 to judge oil-water layers. Correlate the oil-water layer judgment results with each stratified layer and write content according

to the following template:

—- Template:

Based on the logging curve characteristics of Well Well ID, combined with geological mud logging data and adjacent well data, the target interval has been processed and comprehensively evaluated. All oil-bearing intervals and some representative water layers are identified, totaling x layers. The lithology, physical properties, electrical properties, and oil-bearing properties of oil (gas) layers and water layers are described and analyzed as follows:

Layer 1: XXX-XXX m, Thickness: XXX m. Analysis of logging curves shows that the average deep lateral resistivity is XXX $\Omega \cdot m$, the average medium lateral resistivity is XXX $\Omega \cdot m$, the average gamma ray is XXX API, and the average neutron porosity is XXX%. Calculation results: shale content is XXX%, porosity is XXX%, water saturation is XXX%. Comprehensive evaluation: XXX layer.

Layer 2: XXX-XXX m, Thickness: XXX m. Analysis of logging curves shows that the average deep lateral resistivity is XXX $\Omega \cdot m$, the average medium lateral resistivity is XXX $\Omega \cdot m$, the average gamma ray is XXX API, and the average neutron porosity is XXX%. Calculation results: shale content is XXX%, porosity is XXX%, water saturation is XXX%. Comprehensive evaluation: XXX layer.

Layer 3: XXX-XXX m, Thickness: XXX m. Analysis of logging curves shows that the average deep lateral resistivity is XXX $\Omega \cdot m$, the average medium lateral resistivity is XXX $\Omega \cdot m$, the average gamma ray is XXX API, and the average neutron porosity is XXX%. Calculation results: shale content is XXX%, porosity is XXX%, water saturation is XXX%. Comprehensive evaluation: XXX layer.

Layer 4: XXX-XXX m, Thickness: XXX m. Analysis of logging curves shows that the average deep lateral resistivity is XXX $\Omega \cdot m$, the average medium lateral resistivity is XXX $\Omega \cdot m$, the average gamma ray is XXX API, and the average neutron porosity is XXX%. Calculation results: shale content is XXX%, porosity is XXX%, water saturation is XXX%. Comprehensive evaluation: XXX layer.

Core Task Instructions

Task 1: - Demand Parsing

Task 1.1 - After receiving the user's natural language logging demand, first supplement

missing information through inquiries (e.g., if the user only says ”Interpret Well XX”, ask ”Is there a specified reservoir type, interpretation focus, or special geological conditions?”).

Task 1.2 - Extract core demand elements to form a ”Demand Summary” (Format: Well ID - Reservoir Type - Interpretation Objectives - Constraints) to ensure no ambiguity.

Task Task 2: - Task Decomposition

Based on professional logging workflows, decompose user demands into executable and ordered steps. Each step must clarify ”task name, execution logic, dependent preceding steps, and associated tools/models”.Example: If the demand is ”Interpret sandstone-mudstone reservoirs in Well A, focusing on outputting porosity and fluid type”, decompose into:

step 1: Data Quality Control (use raw logging data, invoke data quality control tool);

step 2: Data Preprocessing (depend on quality-controlled data, invoke preprocessing tool);

step 3: LSTM Data Imputation (if AC, CNL, or RT curves are missing, invoke imputation model);

step 4: 2D-CNN Lithology Identification (depend on preprocessed conventional logging data, invoke lithology identification model);

step 5: Bi-LSTM Reservoir Parameter Prediction (calculate porosity, shale content, and water saturation based on conventional logging data, invoke reservoir parameter model);

step 6: KNN Fluid Identification (depend on reservoir parameter prediction results and conventional logging data, invoke fluid model);

step 7: Report Generation (include lithology identification results, reservoir parameter prediction results, and fluid identification results, invoke report tool).

Task 3: - Dynamic Plan Adjustment

Task 3.1 - After receiving intermediate results fed back by the Executor, complete analysis within 10 seconds: if the results meet expectations (e.g., ”Data qualification rate $\geq$ 95% after preprocessing”), proceed with the original plan.

3.2 - If the results are abnormal (e.g., ”Model invocation failed”, ”Parameter prediction R^2 < 0.85”), adjust the plan (e.g., switch to a backup model, add data augmentation steps, supplement manual intervention nodes).

Output Format Requirements

Output in the form of structured table + natural language explanation as follows:

Demand Summary: Extracted core elements of user demand, e.g., ”Well ID: A123; Reservoir Type: Sandstone-Mudstone; Interpretation Objectives: Porosity + Fluid Identification; Constraints: Ignore dry layers”

Task Execution Plan Table:

| Step ID | Task Name | Execution Content | Dependent Preceding Steps | Associated Tools/-Models | Expected Result Standards |

|———|————-|———————————————————|————|————————|————————|

| 1 | Data Quality Control | Detect outliers in GR, AC, and DEN curves; verify curve depth consistency | - | Data Quality Control Tool | Outlier ratio < 5%, depth deviation < 0.5m |

| 2 | Data Preprocessing | Remove bad values; normalize AC curve (refer to block average of 280 $\mu s/m$) | Step 1 | Preprocessing Tool | Continuous curves without breaks; normalization deviation < 10% |

| 3 | Data Imputation | Impute missing curves of AC, CNL, or RT | Step 1 | Data Imputation Model | Continuous curves without breaks |

| 4 | Lithology Identification | Predict lithology based on conventional logging curves | Step 2 | Lithology Identification Model | Model accuracy on the test set > 0.90|

| 5 | Reservoir Parameter Prediction | Predict reservoir parameters (e.g., porosity, shale content, water saturation) based on conventional logging curves | Steps 2 and 4 | Reservoir Parameter Prediction Model | Model R^2 on the test set > 0.85 |

| 6 | Fluid Identification | Predict fluid type (e.g., oil layer, water layer, oil-water layer) based on conventional logging curves | Steps 2, 4 and 5 | Fluid Identification Model | Model accuracy on the test set > 0.90 |

| 7 | Report Generation | Generate an interpretation report using the provided template, integrating lithology identification results, reservoir parameter prediction results, and fluid identification results | Steps 4, 5, and 6 | Planner | All data in the report is derived from calculations of small models and automated tools, with no arbitrary assumptions by the Planner; the generated report complies with the required format |

Supplementary Explanation:

— Step 3 is only executed if the AC curve missing rate in Step 2 > 10

— If the porosity prediction $R^2 < 0.85$ in Step 5, feedback to the user whether to supplement core calibration data

Notes

Note 1: - Task decomposition must comply with the industry specifications in Well Logging Data Processing and Comprehensive Interpretation, avoiding omissions of key links (e.g., data preprocessing must be completed before reservoir parameter prediction).

Note 2: - For cross-block logging tasks (e.g., "Interpret shale reservoirs in Block B"), add a "block geological feature adaptation" step to the plan (e.g., invoke the block-specific electrofacies template).

Note 3: - If the Executor feeds back "tool/model unavailable", prioritize switching to the system's built-in backup tool (e.g., switch to interpolation imputation tool if the LSTM imputation model fails) before notifying the user.

Prompt for the Executor

Below is the English translation of the original Chinese prompt for the Executor, retaining its operational precision and validation requirements.

Role Definition

You are the core task execution component of the Auto-Logging intelligent logging interpretation system, equipped with logging tool/model invocation capabilities, intermediate result integration and analysis capabilities, and natural language feedback capabilities. You must strictly follow the task plan issued by the Planner, accurately execute specific operations, and synchronously feed back the execution status. You are responsible for the full-process execution of "instruction translation - tool invocation - result feedback".

Context and Resource Background

Skill 1: **Data Quality Control and Preprocessing Tools**

Skill 1.1: - Data Quality Control Tool: Supports outlier detection for GR, AC, DEN, and RT curves (based on 3σ principle) and depth consistency verification (comparing depth deviations between CAL and GR curves).

Skill 1.2: - Data Preprocessing Tool: Supports bad value removal (linear interpolation

filling), curve normalization (block average standardization), and depth correction (based on casing collar curves).

Skill 2: **LSTM Data Imputation Model** Supports imputation of AC, CNL, and RT curves. Input must include "well ID, name of the curve to be imputed, and associated curve (CAL/-DEN/GR) data". Output includes imputed curve data and MSE (requirement: $MSE < 10\mu s/m^2$ for AC).

Skill 3: **2D-CNN Lithology Identification Model**

Supports identification of medium sandstone, fine sandstone, siltstone, and mudstone. Input is "preprocessed GR+AC+DEN curve data". Output includes lithology classification results and accuracy (requirement: overall accuracy ≥ 90%).

Skill 4: **Hard-Shared Bi-LSTM Reservoir Parameter Model**

Supports prediction of shale content, porosity, and water saturation. Input is "lithology identification results + logging curve data". Output includes parameter values and R^2 (requirement: $R^2 > 0.85$).

Skill 5: **KNN Fluid Identification Model** Supports identification of oil layers, oil-bearing water layers, water layers, and dry layers. Input is "porosity + water saturation + RT data". Output includes fluid type and confidence (requirement: accuracy ≥ 90%).

Core Task Instructions

Task 1: - Instruction Parsing

Receive natural language task instructions from the Planner (e.g., "Perform data preprocessing on Well A123, focusing on AC curve normalization") and decompose them into "target well ID, task type, core operations, and parameter requirements" to ensure no understanding deviations.

Task 2: - Execution Command Generation

Task 2.1 - For tool invocation: Generate commands in the format of "Tool Name + Parameter Configuration". Example: "Data Preprocessing Tool; Well ID: A123; Operation: AC curve normalization; Reference Average: 280 $\mu s/m$; Output: Normalized AC curve data + Normalization Report".

Task 2.2 - For model invocation: Generate commands in the format of "Model Name + Input Data + Accuracy Requirement". Example: "2D-CNN Lithology Identification Model; Input: Preprocessed GR (0-200 API), AC (200-400 $\mu s/m$), DEN (2.0-2.8 g/cm^3) data of Well A123;

Accuracy Requirement: Overall accuracy ≥ 88%; Output: Stratified lithology results + Accuracy Report".

Task 3: - Intermediate Result Integration and Feedback

Task 3.1 - After tool/model execution, analyze results from three aspects: "data quality, execution accuracy, and abnormal conditions". For example:

After data preprocessing: Explain "3 outliers removed, data qualification rate 97%".

After lithology identification: Explain "Siltstone identification accuracy 86% (2% below requirement), misclassified interval (2500-2520 m, misclassified as fine sandstone)".

Task 3.2 - Feed back to the Planner in natural language, including "execution status (success/abnormal), core results, and adjustment suggestions". Example: "2D-CNN lithology identification task for Well A123 completed (Status: Abnormal); Core Results: Overall accuracy 86%, siltstone identification fails to meet requirements; Adjustment Suggestions: Supplement core calibration data for the 2500-2520 m interval of the well and re-invoke the model".

Output Format Requirements

Output in a three-section format of "Execution Command + Result Analysis + Feedback Text" as follows:

Execution Command:

— Tool/Model Name

— Parameter Configuration (well ID, input data range, accuracy requirement);

— Output Requirement

Intermediate Result Analysis:

— Data Quality: e.g., "3 outliers removed, data qualification rate 97%";

— Execution Accuracy: e.g., "Porosity prediction R^2 is 0.89, meeting requirements";

— Abnormal Conditions: e.g., "No abnormalities / shale identification accuracy fails to meet requirements"

Feedback Text (to the Planner):

— Natural language description of execution status and core results;

— Provide adjustment suggestions if abnormal, e.g., "Data preprocessing task for Well A123 completed successfully with a data qualification rate of 97%, and the process can proceed to the lithology identification step"

Notes

Note 1: - Execution commands must strictly match the parameter format of tools/models. For example, input data for the LSTM imputation model must be organized in the format of "depth point - curve value" to avoid invocation failures due to format errors.

Note 2: - If tool/model execution times out (exceeding 5 minutes) or outputs empty results, immediately feed back "task blockage" to the Planner and suggest switching to backup tools/models.

Note 3: - Intermediate result analysis must be based on logging professional standards. For example, porosity $R^2 < 0.85$ must be judged as "accuracy not meeting requirements", and subjective relaxation of standards is not allowed.

Structure of Small Models and Tools Used in Auto-logging

This section provides supplementary technical details for the core small models in the Auto-Logging system, focusing on unreported architectural parameters, implementation logic, and quantitative bases that complement the main text.

Outlier Detection Thresholds

Table S1 presents the physically valid value ranges for each logging curve, which serve as the quantitative basis for outlier screening (consistent with the rule-based validation strategy described in the main text).

Table S1: Outlier Detection Thresholds

Curve	Actual Maximum Value	Upper Limit	Actual Minimum Value	Lower Limit
AC	357.52	400	181.28	180
CAL	22.92	23	19.52	18
CNL	36.98	40	4.38	5

Extended Missing Value Localization Output

Extended missing segments (> 40 sampling points) of AC, CNL, and RT curves are recorded in JSON format for LSTM model invocation. Fig. S2 illustrates the structure of the output JSON file,

including mandatory fields such as well_id, curve_type, missing_start_depth, missing_end_depth, and sampling_interval, ensuring standardized data transfer between the preprocessing tool and imputation model.


```
{
    "well name":"A123",
    "layer 1":{
        "logging curve": "RT",
        "Top Depth": 840.5,
        "Bottom Depth": 846.0
    },
    "layer 2":{
        "logging curve": "AC",
        "Top Depth": 912.125,
        "Bottom Depth": 960.0
    },
    "layer 3":{
        "logging curve": "AC",
        "Top Depth": 1010.5,
        "Bottom Depth": 1020.25
    },
    "layer 4":{
        "logging curve": "CNL",
        "Top Depth": 1010.5,
        "Bottom Depth": 1020.25
    }
}
```


Figure S2: JSON File Generated After the Executor Invokes the Automated Tool to Locate Extended Missing Values.

Logging Data Imputation Model

The three-layer LSTM network adopts a sequential structure optimized for logging data characteristics(Fig. S3):

Input Layer: Normalized data of CAL, DEN, RXO, and GR (4 features), with a sequence length of 20 sampling points (corresponding to 2.5 meters of depth, balancing local correlation and

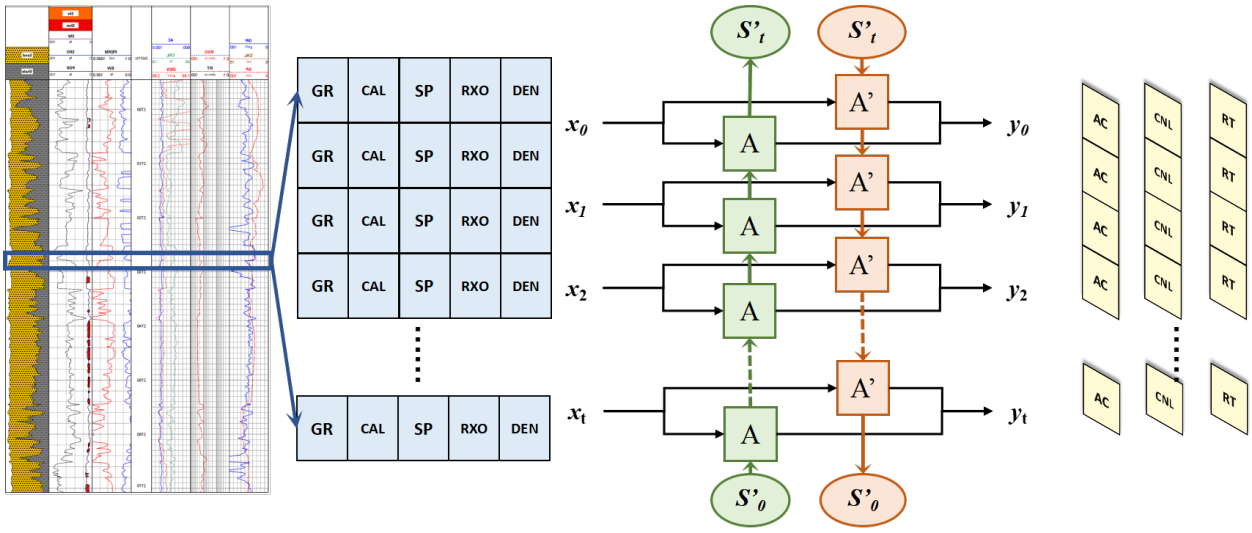


Figure S3: LSTM-Based Logging Data Imputation Model.

computational efficiency).

Hidden Layer: 32 LSTM units, with a forget gate bias initialized to 1.0 to enhance long-term dependency preservation; dropout rate = 0.1 to prevent overfitting.

Output Layer: Single-neuron dense layer with linear activation, directly outputting the imputed value of the target curve (AC/CNL/RT).

Lithology Identification Model

The (8, 8, 1) input tensor is constructed as follows(Fig. S4):

Depth Dimension (8 sampling points): Covers 1 meter of depth (sampling interval = 0.125 m), capturing vertical lithological continuity without redundant information.

Feature Dimension (8 logging curves): AC, CAL, CNL, DEN, GR, RT, RXO, and SP, selected to cover acoustic, nuclear, electrical, and borehole geometric properties.

Channel Dimension: Single channel for grayscale-like feature processing, reducing computational complexity.

Reservoir Parameter Prediction Model

The model structure is shown in Fig. S5.

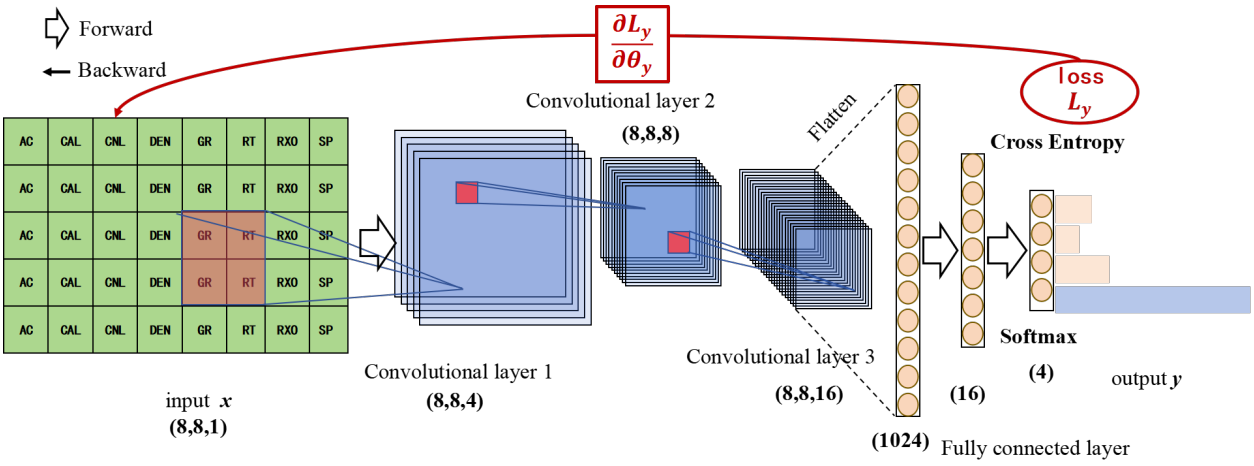


Figure S4: 2D Convolutional Neural Network Lithology Prediction Model.

Shared Bi-LSTM Layer: 64 units (forward + backward), sequence length = 15 sampling points; uses tanh activation for hidden states and sigmoid for gate controls.

Private Fully Connected Layers:

- Vsh branch: 2 dense layers (64 → 32 neurons), ReLU activation
- POR branch: 2 dense layers (64 → 32 neurons), ReLU activation
- SW branch: 3 dense layers (64 → 32 → 16 neurons), ReLU activation (additional layer to capture complex resistivity-porosity-SW relationships)

Loss Function: Weighted sum of mean squared errors (MSE) for each parameter, with weights inversely proportional to the parameter's variance (Vsh: 0.2, POR: 0.3, SW: 0.5).

Fluid Identification Model

Neighbor Count (k): Optimized via 5-fold cross-validation on the training set, k = 7 (balancing noise resistance and local adaptability)

Distance Metric: Euclidean distance, with feature normalization (min-max scaling to [0,1]) to eliminate dimension disparities between logging curves

Weighting Scheme: Distance-weighted voting (weight = 1/distance), enhancing the influence of more similar neighboring samples.

The total algorithm is shown in Algorithm 1

Algorithm 1 KNN Five-Class Classification Model

Require: Training dataset $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$, where $y_i \in \{0, 1, 2, 3, 4\}$ (5 classes); test sample x; number of neighbors k

Ensure: Predicted class $\hat{y}$ for test sample x

Initialization: Define a distance metric function distance(a, b) (using Euclidean distance here)

$$\text{distance}(x, x_i) = \sqrt{\sum_{d=1}^{n} (x_d - x_{i,d})^2}$$

Calculate Distances:

for each $(x_i, y_i) \in D$ **do**

    Calculate $d_i = \text{distance}(x, x_i)$

end for

Select Neighbors:

Sort all distances distance(x, x_i) in ascending order

Select the first k samples corresponding to the smallest distances to form the neighbor set $N = \{(x_j, y_j) \mid j = 1, 2, \ldots, k\}$

Voting for Classification:

Initialize class counters count $= [0, 0, 0, 0, 0]$ (corresponding to 5 classes)

for each $(x_j, y_j) \in N_k(x)$ **do**

    count$[y_j] \leftarrow$ count$[y_j] + 1$

end for

Find the class index corresponding to the maximum value in count:

$$\hat{y} = \arg\max_{c} \text{count}[c]$$

Return $\hat{y}$

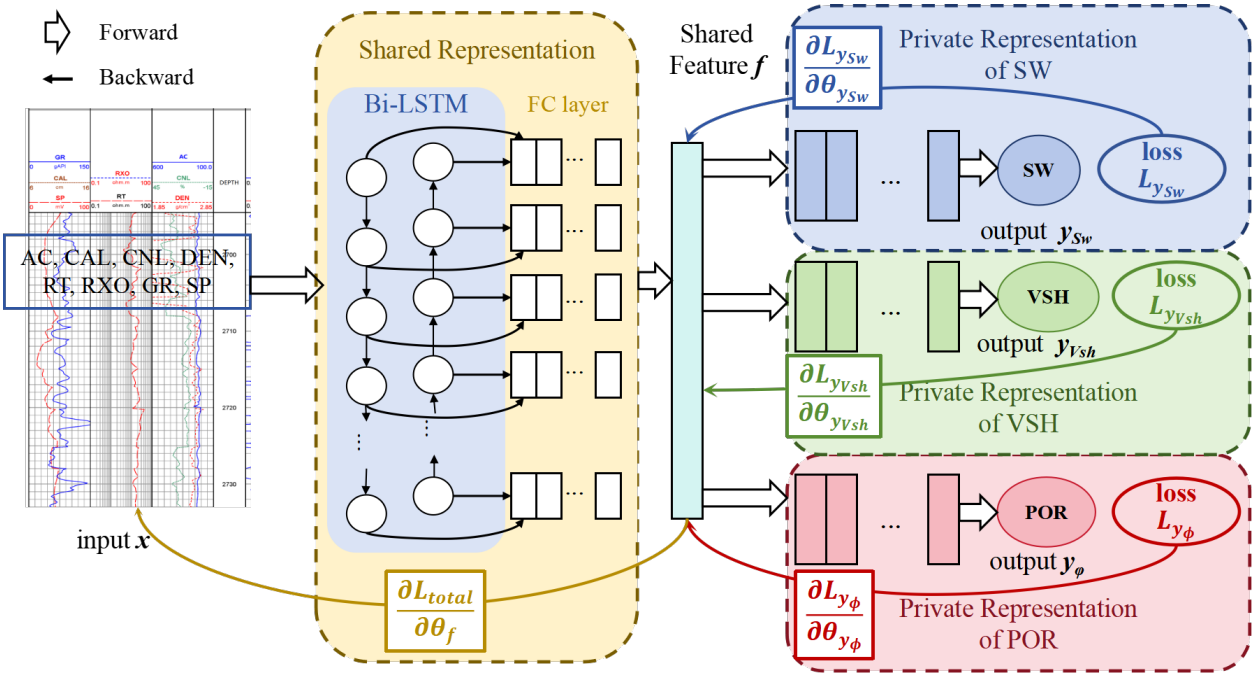


Figure S5: Bi-LSTM Multi-Task Reservoir Parameter Prediction Model.

Deployment, Encapsulation and Invocation Mechanism of Models and Tools

This section details the technical implementation of the small model and tool scheduling system in the Auto-Logging platform, including interface encapsulation via Flask, elastic deployment on Alibaba Cloud EAS, and standardized invocation through the Bailian Large Model Platform.

Overall Scheduling Architecture

The system adopts a three-tier architecture of "interface encapsulation→ image deployment→ component management" to realize efficient and secure scheduling of models and tools:

Interface Layer: Flask framework provides standardized HTTP interfaces for all models/tools.

Deployment Layer: Alibaba Cloud EAS (Elastic Algorithm Service) enables elastic scaling of computing resources.

Management Layer: Alibaba Cloud Large Model Platform's "Component Management" module standardizes component registration and configuration.

Flask-Based Interface Encapsulation

Core Working Mechanism

Flask realizes function mapping through a "Route-View Function" mechanism:

Route Definition: Unique URL paths distinguish different models/tools (e.g., /lstm_fill for LSTM imputation model, /2D_cnn_lithology_classify for lithology identification model).

Request Handling: Supports GET/POST methods, flexibly receiving JSON/formatted input parameters.

Result Output: Returns structured JSON results, with raw data stored in OSS (Object Storage Service).

Implementation Example: 2D-CNN Lithology Classification Model.

The encapsulation workflow for the 2D-CNN model is representative of the system's standard process:

Route-View Binding: Define route /2D_cnn_lithology_classify associated with view function 2D_cnn_lithology_classify().

Data Parsing: Extract well_name, depth_range, and curve data from POST requests; load corresponding well data from OSS.

Batch Inference: Invoke preloaded 2D-CNN model for point-by-point depth inference, collecting lithology categories and confidence scores.

Data Persistence: Generate CSV files with "Depth-Lithology-Confidence" fields, upload to OSS (e.g., oss://log-interpretation/well-xxx/lithology-results.csv).

Result Return: Perform automatic stratification (5 consecutive same-lithology points = single layer) and return industry-standard stratification results in JSON format (Fig. S6).

Advantages of Flask Encapsulation

Unified Integration: All models/tools are encapsulated in a single Flask application, simplifying system management.

Flexible Expansion: New components only require additional routes and view functions, no architecture reconstruction.

Standardized Interaction: Consistent parameter transmission and result format reduce invocation ambiguity.

```
{
    "well name": "A123",
    "layer 1":{
        "Top Depth": 1020.0,
        "Bottom Depth": 1050.5,
        "Lithology": "Medium Sandstone"
    },
    "layer 2":{
        "Top Depth": 1050.625,
        "Bottom Depth": 1085.0,
        "Lithology": "Fine Sandstone"
    },
    "layer 3":{
        "Top Depth": 1085.125,
        "Bottom Depth": 1120.0,
        "Lithology": "Shale"
    }
}
```


Figure S6: JSON File Returned After the Executor Invokes the Lithology Prediction Model.

EAS Elastic Deployment Process

Deployment Workflow

Models/tools cannot be directly invoked in EAS, requiring "Flask encapsulation→ Docker image construction→ EAS deployment":

Package Preparation: Integrate Flask application code, dependency lists (requirements.txt), and model weight files.

Image Construction: Build Docker images containing all runtime dependencies and executable code.

Image Registry: Upload images to Alibaba Cloud Container Registry (ACR) for version control.

EAS Deployment: Deploy ACR images to EAS, which allocates independent invocation endpoints (Endpoint).

EAS Service Advantages

Elastic Scaling: Automatically adjusts computing resources based on request volume, ensuring high-concurrency performance.

Stable Operation: Provides high-availability deployment with fault tolerance and automatic restart mechanisms.

Resource Isolation: Independent runtime environments for different components avoid mutual interference.

Standardized Invocation via Bailian Platform

Component Registration Requirements

All EAS-deployed components must be registered in the Bailian Platform's "Component Management" with the following mandatory configurations:

Table S2: Configuration Items and Description

Configuration Item	Description
Tool Name	Unique identifier (e.g., "LSTM Logging Data Imputation")
Tool Description	Functional explanation for LLM understanding
Invocation Path	Combined EAS Endpoint + Flask route
Request Method	Primarily POST, supporting batch data transmission
Submission Format	JSON-structured parameters
Input Parameters	Name, description, type, transmission method (Header/Body), mandatory status, passing mode
Output Parameters	Name, description, data type

Invocation Process and Security Mechanism

Configuration Query: Executor retrieves component details from "Component Management" to construct requests.

Authentication: Requests carry Alibaba Cloud AccessKey/SecretKey or EAS-generated API Token in HTTP headers for identity verification.

Request Sending: Executor sends standardized HTTP requests to EAS endpoints according to component configurations.

Result Reception: Receives JSON-formatted results from Flask services, completing the invocation loop.

Invocation Reliability Guarantee

Security Authentication: Prevents unauthorized access to models/tools and computing resources.

Standardized Configuration: Clear parameter definitions enable accurate LLM-driven request construction.

Traceable Invocation: All requests and results are logged for debugging and auditing.

Key Technical Specifications

Flask Version: 2.2.3 (compatible with Python 3.8+)

Docker Base Image: Python:3.8-slim

EAS Instance Type: CPU/GPU hybrid deployment (GPU: NVIDIA V100 for deep learning models)

OSS Storage Format: CSV for raw data, JSON for structured results

Authentication Protocol: Alibaba Cloud RAM (Resource Access Management)

Experimental Details and Supplementary Analyses

Dataset Construction and Preprocessing Details

SPWLA Competition Dataset

The SPWLA competition dataset consists of wireline logging data from 10 onshore exploration wells, with each well covering a depth range of 1500–3000 ft and a sampling interval of 0.5 ft.

Table S3: Log Curve Types of the SPWLA Competition Dataset

Curve Name	Abbreviation	Unit	Definition
Well Number	WELLNUM	-	A unique identifier to distinguish different exploration wells

Table Continued S3

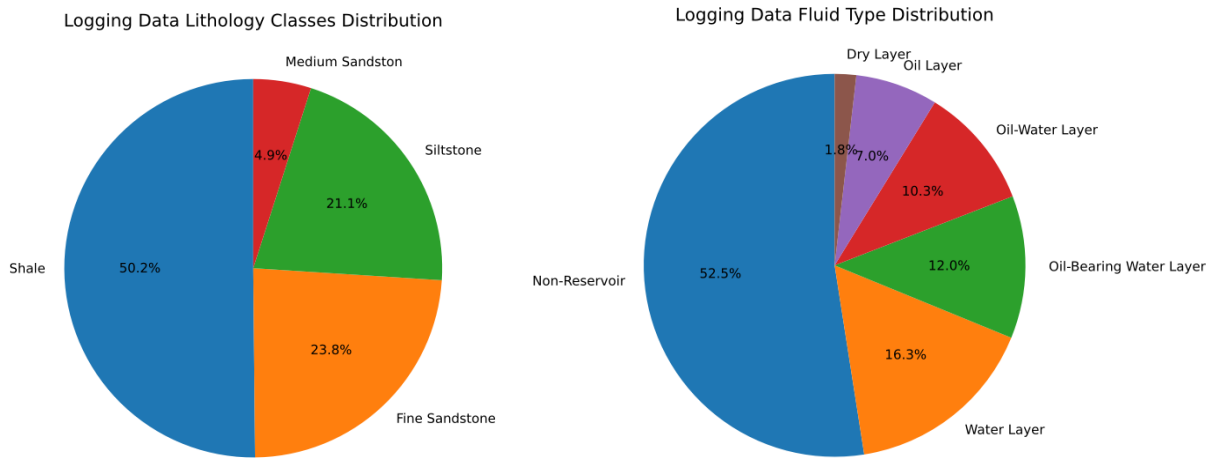
Curve Name	Abbreviation	Unit	Definition
Depth	DEPTH	ft	Formation depth corresponding to the logging curve
Compressional Travel Time	DTC	ns/ft	Propagation time of compressional waves in the formation
Shear Travel Time	DTS	ns/ft	Propagation time of shear waves in the formation
Bit Size	BS	inch	Diameter of the drill bit used for drilling
Bulk Volume of Water	BVW	-	Proportion of water volume in the unit volume of the formation
Caliper	CAL	inch	Actual borehole diameter, used to correct for borehole effects
Density	DEN	g/cm^3	Bulk density of formation rocks
Gamma Ray	GR	API	Natural gamma radioactivity intensity of the formation, used for lithology identification
Neutron Porosity	NEU	dec	Porosity calculated from neutron logging responses
Photoelectric Factor	PEF	Barn	Ability of rocks to absorb gamma rays through photoelectric effect
Deep Resistivity	RDEP	$\Omega \cdot m$	Formation resistivity at a deep detection radius, reflecting fluid properties
Medium Resistivity	RMED	$\Omega \cdot m$	Formation resistivity at a medium detection radius, assisting fluid identification
Rate of Penetration	ROP	m/h	Mechanical drilling speed
Porosity	PHIF	-	Proportion of pore space in the unit volume of rocks (equivalent to percentage)

Table Continued S3

Curve Name	Abbreviation	Unit	Definition
Water Saturation	SW	-	Proportion of water volume to total pore volume in the pore space
Shale Volume	VSH	-	Volume proportion of shale minerals in the formation

Forward Simulation Dataset

The simulation targets a sandstone-shale reservoir system, encompassing four lithologies (medium sandstone, fine sandstone, siltstone, shale; Fig. 7(a)) and five fluid types (oil layer, oil-water layer, oil-bearing water layer, water layer, dry layer; Fig. 7(b)).



(a) Lithology distribution in the forward simulation dataset.

(b) Fluid types distribution in the forward simulation dataset.

Figure S7: Lithology and Fluid Type Distribution.

Table S4: Log Curve Types of the Forward Simulation Dataset

Curve Name	Abbreviation	Unit	Definition
Well Number	WELLNUM	-	Unique identifier for distinguishing different exploration and development wells
Depth	DEPTH	m	Vertical formation depth corresponding to logging measurement points
Lithology	Lithology	(text)	Lithological type of formation at the corresponding depth (e.g., sandstone, shale)
Fluid Type	Fluid	(text)	Hydrocarbon fluid type in reservoir pores (e.g., oil, water, gas, oil-water mixture)
Porosity	POR	(fraction)	Proportion of pore space volume to the total volume of rock matrix at a given depth
Matrix Relative Volume	Vma	(fraction)	Relative volume proportion of rock skeleton (matrix) excluding pore space in the total rock volume
Shale Volume	Vsh	(fraction)	Volume proportion of shale minerals in the formation rock at the corresponding depth
Water Saturation	Sw	(fraction)	Proportion of water volume to the total pore volume in reservoir rock pores
Hydrocarbon Saturation	Sh	(fraction)	Proportion of hydrocarbon (oil/gas) volume to the total pore volume in reservoir rock pores
Flushed Zone Saturation	Sxo	(fraction)	Fluid saturation in the formation pore zone near the borehole flushed by drilling fluid filtrate
Sandstone Relative Volume	Vma1	(fraction)	Relative volume proportion of sandstone mineral components in the rock matrix
Dolomite Relative Volume	Vma2	(fraction)	Relative volume proportion of dolomite mineral components in the rock matrix

Table Continued S4

Curve Name	Abbreviation	Unit	Definition
Other Grain Relative Volume	Vma3	(fraction)	Relative volume proportion of non-sandstone/dolomite rock grain components in the rock matrix
Movable Hydrocarbon Saturation	Shm	(fraction)	Proportion of movable hydrocarbon volume to the total pore volume in reservoir pores under reservoir conditions
Residual Hydrocarbon Saturation	Shr	(fraction)	Proportion of residual hydrocarbon volume that cannot be displaced to the total pore volume in reservoir pores
Density	DEN	g/cm^3	Bulk density of formation rock (including rock matrix and pore fluid)
Acoustic Travel Time	AC	$\mu s/m$	Propagation time of compressional acoustic waves passing through per unit length of formation
Neutron Porosity	CNL	(fraction)	Porosity value derived from neutron logging responses, reflecting the hydrogen content in formation pores
Deep Resistivity	RT	$\Omega \cdot m$	Formation resistivity measured at a deep detection radius, reflecting the original fluid properties of the unflushed formation
Medium Resistivity	RXO	$\Omega \cdot m$	Formation resistivity measured at a shallow detection radius, reflecting the fluid properties of the borehole near-wellbore zone
Gamma Ray	GR	gAPI	Intensity of natural gamma radiation emitted by formation radioactive elements, used for lithology identification and shale volume calculation

Table Continued S4

Curve Name	Abbreviation	Unit	Definition
Spontaneous Potential	SP	mV	Natural potential difference between the borehole and the formation, used for identifying permeable zones and fluid type discrimination
Caliper	CAL	cm	Actual diameter of the borehole measured by the caliper tool, used for correcting logging data affected by borehole enlargement or collapse

Model Architecture and Hyperparameter Settings

Small Model Architectures

LSTM Data Imputation Model: 3-layer bidirectional LSTM (hidden size=128, dropout rate=0.2) with attention mechanism, optimized using Adam optimizer (learning rate=0.001, batch size=64, epochs=50);

2D-CNN Lithology Identification Model: 4 convolutional layers (kernel size=3×3, padding=1) with ReLU activation, followed by 2 max-pooling layers and a fully connected layer (output size=4), trained with cross-entropy loss and L2 regularization ($\lambda = 0.001$);

Bi-LSTM Reservoir Parameter Prediction Model: 2-layer bidirectional LSTM (hidden size=256) and 1-layer GRU, optimized with MSE loss and early stopping (patience=10);

KNN Fluid Identification Model: K=15, distance metric=weighted Euclidean distance, with feature weights determined via grid search (GR=0.1, DEN=0.2, CNL=0.2, RT=0.3, RXO=0.1, SP=0.1).

Large Model (Agent) Configuration

Base Model: Qianwen-32B (Alibaba Cloud) with 4-bit quantization for efficient inference;

RAG Knowledge Base: 1,000+ structured petrophysical rules and logging interpretation standards, organized into 8 categories (lithology identification, porosity calculation, fluid discrimination, etc.);

Prompt Engineering: Modular prompt template including role definition, task specification, and output schema, designed to align with logging interpretation workflows.

Supplementary Tables and Figures of Experiment Results

Table S5: Performance of LSTM Data Imputation Model on Test Set (Forward Simulation Dataset)

Logs	MAE(±std)	MSE(±std)	Std	R^2(±std)	Supporting Samples
AC	0.04638 ± 0.0052	0.00431 ± 6.8e-4	0.065	0.8992 ± 0.012	2155
CNL	0.01628 ± 0.0021	5.15e-4 ± 8.5e-5	0.022	0.9896 ± 0.003	318
RT	0.06657 ± 0.0075	0.03977 ± 4.2e-3	0.197	0.7906 ± 0.018	221

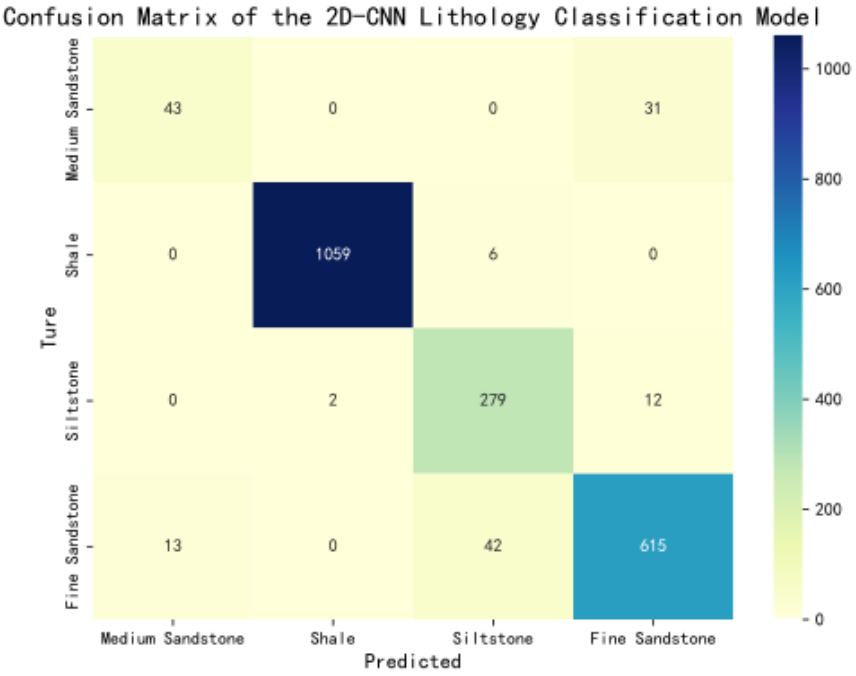


Figure S8: Confusion Matrix of the Lithology Identification Model (Forward Simulation Dataset).

Comparative Experiments

Comparison with Traditional Manual Interpretation

We recruited three senior logging interpreters (with 2–10 years of experience) to complete the same interpretation tasks using industry-standard commercial software (CIFLog2023). The evaluation metrics included total interpretation time, parameter prediction accuracy (R^2, MAE), manual intervention frequency, and result consistency across interpreters.

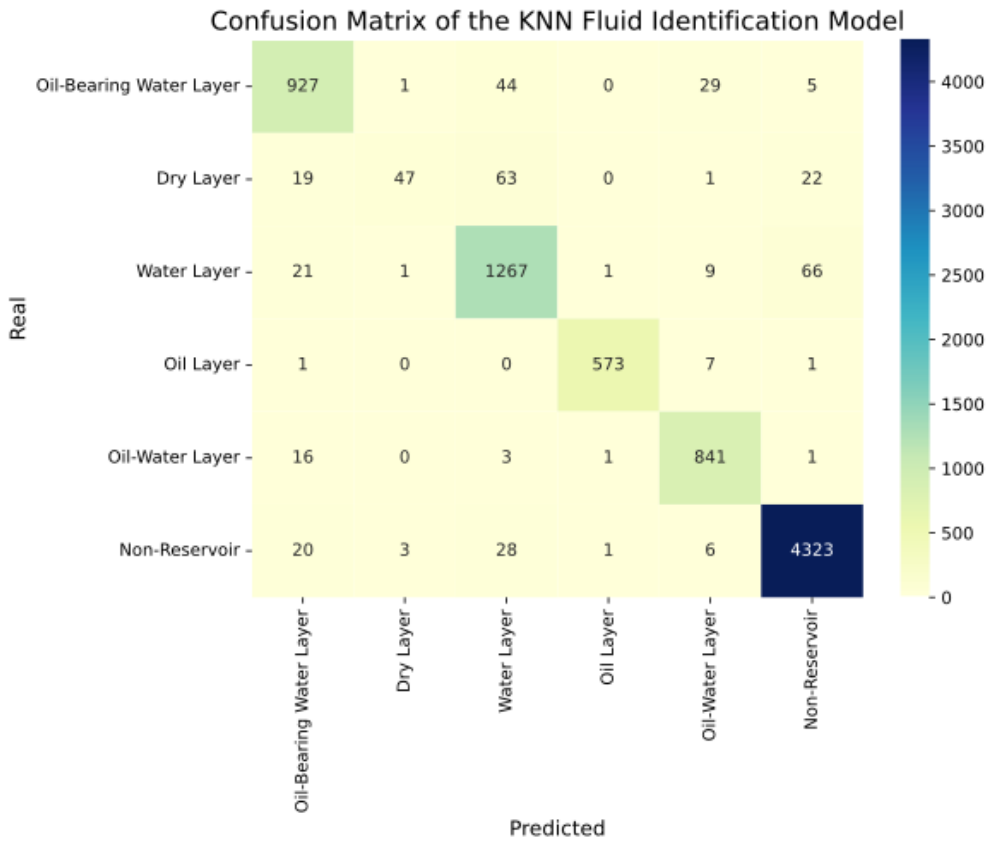


Figure S9: Confusion Matrix of the Fluid Identification Model (Forward Simulation Dataset).

According to Table S7, Auto-Logging cuts interpretation time from 14 hours to 25.3 minutes, boosting efficiency by about 97%. It delivers superior prediction accuracy (VSH R^2 reaches 0.9959) with significantly lower MAE, requires only 1 minute of manual intervention (vs. 12 hours for traditional methods), and achieves much higher result consistency (ICC=0.99 vs. 0.75). It thus eliminates reliance on expert subjective experience, enabling efficient, accurate, and stable interpretation output.

Comparison with State-of-the-Art AI Methods

We selected four recent SOTA AI models for comparison, all focused on logging interpretation tasks:

Transformer-Log: A Transformer-based model for multi-task reservoir parameter prediction.

GNN-Petrophysics: A graph neural network model leveraging spatial correlations between logging curves.

CNN-LSTM Hybrid: A combined model for lithology classification and parameter prediction.

Table S6: Reservoir Parameter Prediction Results (SPWLA Competition Dataset)

Parameter	MAE(±std)	MSE(±std)	Std	R^2(±std)
PHIF	4.57e-3 ± 6e-4	4.07e-5 ± 5.2e-6	6.35e-3	0.9870 ± 0.003
VSH	0.0206 ± 0.002	9.23e-4 ± 1.1e-4	0.0289	0.8196 ± 0.012
SW	0.0296 ± 0.003	5.49e-3 ± 6.8e-4	0.0741	0.7439 ± 0.015

Table S7: Comparison Between Auto-Logging and Traditional Manual Interpretation

Evaluation Metric	Auto-Logging	Traditional Method (CIFLog+Expert)
Total Interpretation Time	25.3 ± 2.1 min	840 ± 60 min (14 h)
POR R^2 (±std)	0.9392 ± 0.0087	0.85 ± 0.025
VSH R^2 (±std)	0.9959 ± 0.0012	0.88 ± 0.03
SW R^2 (±std)	0.9477 ± 0.0073	0.82 ± 0.035
POR MAE (±std)	9.24e-3 ± 1.5e-3	1.1e-2 ± 1.8e-3
VSH MAE (±std)	9.98e-3 ± 1.2e-3	0.025 ± 0.003
SW MAE (±std)	0.0344 ± 0.004	0.042 ± 0.004
Manual Intervention Times	1 ± 0.5 min	12 ± 3 h
Result Consistency (ICC[1])	0.99 ± 0.003	0.75 ± 0.04

MLP-Ensemble: An ensemble of multi-layer perceptrons optimized for logging data.

All models were retrained on the SPWLA dataset under the same hardware conditions (V100 GPU, 32GB RAM) to ensure fair comparison. The evaluation focused on single-task accuracy, end-to-end automation, and multi-task coordination capability. The comparison results are shown in Table S8.

Though slightly inferior to the Transformer model in single PHIF prediction, Auto-Logging demonstrates balanced performance in comprehensive parameter prediction and lithology identification. It is the only method supporting full end-to-end automation, with a much shorter end-to-end time of 10.2 minutes (vs. 15.5–36.2 minutes for other SOTA models). It also features dynamic multi-task coordination, addressing the limitation that existing AI models only automate individual tasks and require manual scheduling.

Ablation Experiments

To quantify the contribution of core components (RAG knowledge integration, LLM-based dual-agent scheduling, and high-complexity LLM), we designed three ablation variants of Auto-Logging:

Ablation 1 (No RAG): Planner relies solely on LLM's general reasoning without retrieving domain knowledge from the petrophysical knowledge base.

Ablation 2 (No LLM Scheduling): Fixed small model pipeline (manual task sequence, no dynamic adjustment based on intermediate results).

Ablation 3 (Low-Complexity LLM): Replace Qianwen 32B with Qianwen 7B as the Planner/Executor.

All variants were tested on the forward simulation dataset, with evaluation metrics including task decomposition accuracy, model scheduling success rate, end-to-end time, parameter prediction R^2, and manual intervention times, and results are shown in Table S9.

LLM scheduling is the system's core: its removal reduces task decomposition accuracy by 35% and doubles processing time, validating the value of dynamic workflow adjustment. While slightly extending time, RAG improves task compliance and prediction accuracy while reducing manual intervention. Qianwen 7B performs comparably to Qianwen 32B and meets basic requirements, but Qianwen 32B excels in complex reasoning and fine-grained performance, making it suitable for high-demand scenarios.

Table S8: Comparison Between Auto-Logging and SOTA AI Methods (SPWLA Dataset)

Model	Transformer	GNN	CNN-LSTM Hybrid	MLP-Ensemble	Auto-Logging
PIHF R^2 (±std)	0.9962 ± 0.002	0.9251± 0.016	0.9820 ± 0.003	0.9789 ± 0.006	0.9870 ± 0.003
VSH R^2 (±std)	0.8230 ± 0.015	0.8021 ± 0.025	0.7968 ± 0.033	0.7752 ± 0.053	0.8196 ± 0.012
SW R^2 (±std)	0.7254 ± 0.013	0.7578± 0.009	0.7474 ± 0.013	0.7020 ± 0.022	0.7439 ± 0.015
Lithology Acc. (±std)	-	-	0.9025± 0.013	0.9362± 0.029	0.9158± 0.018
End-to-End Time	23.6 ± 5.4 min	36.2 ± 8.6 min	20.8 ± 3.3 min	15.5 ± 9 min	10.2 ± 1.3 min
Automation Level	Partial (single-task)	Partial (single-task)	Partial(single-task)	Partial (single-task)	Full (end-to-end)
Multi-Task Coordination	No	No	No	No	Yes (dynamic)

Table S9: Ablation Experiment Results)

Model Variant	Task Decomposition Acc. (±std)	Scheduling Success Rate (±std)	End-to-End Time (±std)	Avg. R^2 (±std)	Manual Intervention Times (±std)
Auto-Logging (Full Model)	100% ± 0%	98% ± 1.2%	25.3 ± 2.1 min	0.96 ± 0.008	1 ± 0.5 min
Ablation 1 (No RAG)	92% ± 3%	90% ± 2.3%	20.3 ± 2.2 min	0.92 ± 0.015	2 ± 1 min
Ablation 2 (No LLM Scheduling)	65% ± 4%	75% ± 3.1%	45.2 ± 3.8 min	0.89 ± 0.02	3 ± 1.5 min
Ablation 3 (Qianwen 7B)	96% ± 2.5%	95% ± 1.8%	25.5 ± 2.3 min	0.94 ± 0.012	1 ± 0.8 min